跨境电商业务

基于前后端分离的跨境电商独立站系统的设计与实现

基于前后端分离的跨境电商独立站系统的设计与实现

- 第一章 引言
  - 1.1 研究背景
  - 1.2 研究意义
  - 1.3 国内外研究现状
  - 1.4 研究内容与目标
  - 1.5 论文组织结构
- 第二章 可行性分析
  - 2.1 技术可行性
  - 2.2 经济可行性
  - 2.3 操作可行性
  - 2.4 社会可行性
  - 2.5 可行性分析小结
- 第三章 系统需求分析
  - 3.1 系统概述与需求分析目标
  - 3.2 用户角色分析
  - 3.3 功能需求分析
    - 3.3.1 管理端功能需求

第一章 引言

1.1 研究背景

随着互联网与移动终端的普及，电子商务已深度融入社会生产与消费体系。传统依托第三方平台的店铺模式在流量成本、数据归属和品牌自主性等方面存在明显局限，越来越多的企业转向搭建**独立站**（Independent E-commerce Site），以拥有自有域名、自主运营和完整数据闭环的在线销售渠道。在跨境贸易场景下，独立站能够直接面向海外消费者，配合 PayPal 等国际支付方式，成为跨境电商的重要形态之一。

独立站系统通常包含**前台商城**（面向终端用户浏览、加购、下单与支付）、**管理后台**（面向运营与管理人员进行商品、订单、内容与系统配置管理）以及**后端 API 服务**（为前后端提供统一的数据与业务支撑）。为适应多端接入、快速迭代与团队协作，采用**前后端分离**架构已成为主流选择：前端负责展示与交互，后端专注业务逻辑与数据持久化，通过 RESTful API 进行通信。在此背景下，设计并实现一套结构清晰、功能完整、便于扩展的跨境电商独立站系统，具有明确的工程应用价值，也适合作为计算机软件工程领域的毕业设计课题。

1.2 研究意义

本课题的研究意义主要体现在以下方面：

（1）工程实践意义。 通过从需求分析、架构设计到编码实现的全流程实践，将软件工程中的分层架构、模块化设计、接口规范与安全认证等知识落到实处。系统后端采用 Go 语言与 Gin 框架，实现 Handler → Service → Repository 的分层结构，并引入工厂模式统一管理仓储与业务服务，有利于理解企业级后端开发的组织方式；前端分别采用 Vue 3 + Vite（管理后台）与 Nuxt 4（前台商城），涵盖 SPA 与 SSR/预渲染等不同形态，对现代前端技术栈形成完整认知。

（2）业务完整性。 系统覆盖独立站的核心业务闭环：商品与分类、SKU 与属性、购物车与订单、PayPal 支付与退款、用户与地址、CMS 文档与推荐位、站点与市场配置等。管理端与客户端双端协同，既能支撑日常运营，又能为终端用户提供从浏览、加购到结算、支付的完整购物流程，具备可演示、可部署的完整业务价值。

（3）技术代表性。 技术选型具有较好的代表性：后端使用 Go + Gin + GORM + MySQL + Redis，兼顾性能与开发效率；认证方面区分管理员（Redis Session）与前台用户（JWT），并引入设备指纹支持游客购物车与订单关联；支付对接 PayPal，符合跨境场景需求。前端管理后台采用 Ant Design Vue、Pinia、ECharts 等成熟生态，前台商城采用 Nuxt UI、Tailwind CSS 等现代方案，对当前主流技术栈具有较好的覆盖度。

（4）教学与答辩价值。 项目结构清晰、文档齐全，各子项目（Go 后端、Nuxt 前台商城、Vue 管理后台）职责明确，便于在毕业论文中按“总体设计—后端设计—前端设计—实现与测试”的逻辑展开论述，适合作为计算机软件工程专业毕业设计的范本。

1.3 国内外研究现状

国外方面，电子商务平台与独立站建站技术已较为成熟。Shopify、WooCommerce、Magento 等产品在独立站搭建、主题与插件生态、支付与物流对接等方面形成了完整方案；技术架构上，微服务、前后端分离、Headless CMS 等模式被广泛采用。学术界与工业界在电商系统的高可用、安全支付、推荐算法、用户体验优化等方面持续有大量研究与落地实践。

国内方面，跨境电商与独立站建设受到政策与市场双重推动。技术路线多采用 Java/Go/Node 等语言构建后端服务，Vue/React 等框架构建前端，配合 MySQL、Redis、OSS 等中间件。国内对独立站的研究多集中在运营策略、流量获取、支付与合规等方面；在工程实现上，前后端分离、RESTful API 设计、JWT 与多端认证等已成为常见实践。本课题在技术选型与架构设计上吸收上述主流方案，结合 Go 语言高性能与 Nuxt/Vue 生态，实现一套可运行、可扩展的独立站系统。

1.4 研究内容与目标

本课题的研究内容与目标如下：

（1）系统总体设计。 确定跨境电商独立站的整体架构，包括后端 API 服务、管理后台与前台商城三个子系统的划分，以及前后端分离下的接口规范、认证方式（管理员 Session、用户 JWT、设备指纹）与数据流向。

（2）后端服务的设计与实现。 基于 Go + Gin 构建统一后端 API，采用 Handler → Service → Repository 分层架构，使用 GORM 操作 MySQL、Redis 管理会话与缓存，实现核心系统（管理员、部门、角色、权限、配置）、商品与店铺（商品、分类、属性、标签、SKU）、订单与支付（订单、退款、PayPal）、用户与端侧（用户、地址、购物车）、内容与运营（文档、推荐位）以及资源与配置（OSS、站点与市场设置）等模块，并为管理端与客户端提供清晰的 API 分组（如 `/api/manage`、`/api/client`）。

（3）管理后台的设计与实现。 基于 Vue 3 + TypeScript + Vite 与 Ant Design Vue 构建 shop_admin 管理后台，实现仪表板、商品管理、订单管理、内容管理（CMS）、站点管理与系统管理等功能，满

足商家/管理员对商品、订单、文档、推荐位及系统配置的日常维护需求。

（4）前台商城的设计与实现。 基于 Nuxt 4 + Vue 3 + Nuxt UI 构建 zhuyi-store 前台商城，实现首页、分类集合、商品详情、购物车、结算、订单详情、博客/文档、登录注册与地址管理等页面与流程，支持设备指纹下的游客购物车与订单查询，并完成与后端 API 及 PayPal 支付的对接。

（5）系统联调与功能验证。 对管理端、客户端与后端进行联调，验证商品上下架、订单创建与状态流转、PayPal 支付与退款、用户注册登录、CMS 内容展示等核心流程，确保系统可部署、可演示并满足毕业设计答辩要求。

通过以上工作，最终形成一套**功能完整、架构清晰、技术栈主流**的跨境电商独立站系统，并在此基础上完成毕业论文的撰写与答辩。

1.5 论文组织结构

本论文共分为七章，各章内容安排如下：

- **第一章 引言**：阐述研究背景、意义、国内外研究现状、研究内容与目标以及论文组织结构。
- **第二章 可行性分析**：从技术、经济、操作与社会四个方面论证项目实施的可行性。
- **第三章 系统需求分析**：从业务需求与功能需求出发，分析管理端、客户端及后端 API 的用例与约束，给出功能模块划分与接口需求概要。
- **第四章 系统总体设计（概要设计）**：描述系统总体架构、前后端分离与通信方案、子系统的划分与职责、认证与安全设计、数据库总体设计与接口总体设计。
- **第五章 详细设计**：详述后端分层架构（Handler、Service、Repository、Factory）、路由与中间件、核心业务模块（core、商品与店铺、订单与支付、用户与端侧、CMS、资源）及关键接口与数据模型；并补充管理后台（shop_admin）与前台商城（zhuyi-store）的详细设计，包括技术选型、模块与页面结构、主要组件与流程设计、API 对接方式。
- **第六章 系统实现**：从后端服务、管理后台与前台商城三个子系统分别阐述实现要点，包括入口与配置、路由与中间件、主要页面与组件、API 对接与全局状态管理，体现从设计到可运行系统的实现过程。
- **第七章 系统测试与总结**：给出主要功能与流程的测试方案与结果，总结本课题的工作、不足与后续改进方向。

以上各章围绕“需求—设计—实现—测试”的软件工程主线展开，突出本独立站系统在架构、业务与技术实现上的完整性与规范性。

第二章 可行性分析

可行性分析是在系统正式开发前，从技术、经济、操作等方面评估项目是否具备实施条件的过程。本章从技术可行性、经济可行性、操作可行性和社会可行性四个方面对本跨境电商独立站系统进行分析，论证课题实施的可行性。

2.1 技术可行性

技术可行性主要分析所选技术栈是否成熟、稳定，能否支撑系统功能与性能要求，以及开发团队是否具备相应的技术储备。

（1）后端技术栈。 本系统后端采用 Go 1.24+ 与 Gin 框架，配合 GORM 操作 MySQL、go-redis 操作 Redis。Go 语言由 Google 维护，语法简洁、并发模型成熟，在云原生与高并发场景中应用广泛；Gin 是 Go 生态中主流的轻量级 Web 框架，路由与中间件机制完善，文档与社区活跃；GORM 是成熟的 ORM 库，支持迁移、关联与事务，与 MySQL 配合稳定。Redis 用于会话与缓存，可满足管理员 Session 与高并发读写的需求。上述技术均为开源、文档齐全，在工业界有大量成功案例，**技术成熟度与稳定性可满足本系统要求**。

（2）支付与第三方集成。 系统支付模块对接 PayPal，采用成熟的 plutov/paypal SDK，支持创建订单、捕获订单与退款等标准流程，并可通过配置切换沙箱与生产环境。PayPal 提供完整的 REST API 与文档，对接方式明确，**支付集成在技术上是可行的**。邮件发送采用 gomail，JWT 采用 dgrijalva/jwt-go 等常用库，均为成熟方案。

（3）前端技术栈。 管理后台采用 Vue 3 + TypeScript + Vite + Ant Design Vue 4.x + Pinia。Vue 3 与 Vite 是当前主流的前端组合，构建速度快、生态完善；Ant Design Vue 提供完整的后台 UI 组件与表格、表单、图表等能力，可显著提升开发效率。前台商城采用 Nuxt 4 + Vue 3 + Nuxt UI 4 + Tailwind CSS，Nuxt 4 支持 SSR 与静态预渲染，有利于 SEO 与首屏性能；Nuxt UI 与 Tailwind 可快速搭建响应式页面。上述技术均为业界主流，文档与社区资源丰富，**前端在技术上是可行的**。

（4）架构与接口。 系统采用前后端分离、RESTful API 设计，后端按 Handler → Service → Repository 分层，通过工厂模式统一管理仓储与业务服务。该架构模式在软件工程中已被广泛验证，易于扩展与测试。管理端与客户端 API 分组清晰（`/api/manage`、`/api/client`），认证方式明确（管理员 Redis Session、前台用户 JWT、设备指纹），**架构设计在技术上是可行的**。

（5）开发环境与工具。 开发所需环境为：Go 1.24+、Node.js（推荐 LTS）、MySQL、Redis、Git。上述软件均可免费获取，在 Windows / macOS / Linux 上均可部署；IDE 可采用 VS Code、GoLand 等，对毕业设计阶段的个人或小组开发而言，**开发环境易于搭建，技术可行性高**。

综上，本系统所采用的技术栈成熟、稳定，架构清晰，第三方集成有现成方案，**从技术角度具备可行性**。

2.2 经济可行性

经济可行性主要分析项目在开发、部署与维护过程中的成本是否可接受，以及是否具备必要的资源条件。

（1）开发成本。 本课题为毕业设计项目，开发主体为学生本人或小组。所采用的技术均为开源技术：Go、Gin、GORM、Vue、Nuxt、Ant Design Vue、Nuxt UI、MySQL、Redis 等均无需授权费用。PayPal 提供沙箱环境供开发与测试，无需在开发阶段产生真实支付费用。开发所需硬件为普通 PC 或笔记本，无需额外采购高性能服务器。因此，**开发阶段的经济成本主要为时间与人力成本，无显著软件授权或硬件投入**，对毕业设计而言经济上可行。

（2）部署与运行成本。 系统可部署于单机或云主机。若采用云服务，可选择按量计费或低配包年套餐（如 1 核 2G 云服务器 + MySQL/Redis 托管或自建），在演示与答辩阶段成本可控。若仅在本机部署用于答辩演示，则无额外运行费用。**部署与运行成本在毕业设计可接受范围内**。

（3）维护与扩展成本。 系统采用分层与模块化设计，代码结构清晰，便于后续维护与功能扩展。技术栈主流，社区资源丰富，遇到问题时可较快获得文档或社区支持，**长期维护与扩展的经济与技术成本可控**。

综上，本课题在开发、部署与维护方面无需高额投入，**经济上可行**。

2.3 操作可行性

操作可行性主要分析系统建成后是否便于使用、管理与维护，以及是否与用户能力和组织流程相适应。

（1）管理后台操作。 管理后台（shop_admin）面向商家或运营人员，采用 Ant Design Vue 提供的表格、表单、树形结构等组件，界面布局清晰，功能按模块划分（仪表板、商品管理、订单管理、内容管理、站点管理、系统管理）。商品录入、订单处理、文档与推荐位配置等操作均有明确入口与交互流程，**对具备基本计算机操作能力的用户而言，学习成本低，操作可行**。

（2）前台用户操作。 前台商城（zhuyi-store）面向终端消费者，页面结构符合常见电商习惯：首页、分类、商品详情、购物车、结算、订单查询、登录注册、地址管理等。支持游客加购与按设备指纹查询订单，降低未注册用户的使用门槛。**用户无需专门培训即可完成浏览、加购、下单与支付等操作，操作可行**。

（3）系统部署与运维。 后端通过 YAML 配置文件管理数据库、Redis、邮件、PayPal 等参数，部署时仅需修改配置并启动可执行文件；前端为静态资源或 Node 服务，可单独构建与部署。数据库采用 MySQL，可使用常规备份与恢复手段。**对具备基本运维知识的开发者而言，部署与日常维护操作可行**。

（4）开发与调试。 前后端分离后，前端与后端可并行开发与联调，通过统一 API 基地址与 CORS 配置即可在本地完成联调。**从开发与调试角度，操作流程清晰，操作可行性高**。

综上，**本系统在管理端、用户端以及部署运维方面的操作均具备可行性**。

2.4 社会可行性

社会可行性主要分析项目是否符合法律法规、社会习惯与伦理规范，以及是否存在较大的社会风险。

（1）合规性。 本系统为教学与毕业设计用途的电商独立站，不涉及违禁品交易。支付采用 PayPal，遵循其服务条款与当地金融监管要求；用户注册与隐私数据处理在系统设计中可通过隐私政策与用户协议加以规范。**在遵守学校规定与一般互联网服务规范的前提下，项目在社会与法律层面无显著合规障碍**。

（2）应用场景。 独立站与跨境电商是政策鼓励的发展方向，技术路线与业务模式符合行业趋势。本课题所实现的系统可用于课程演示、毕业答辩以及作为个人作品展示，**社会接受度高，应用场景明确**。

（3）伦理与安全。 系统在设计中采用密码加密存储、JWT 与 Session 认证、设备指纹区分用户等常见安全措施，不涉及对用户数据的滥用设计。**在正常使用与演示范围内，社会可行性与伦理风险可控**。

综上，**本课题在社会、法律与伦理方面具备可行性**。

2.5 可行性分析小结

维度	结论	主要依据
技术可行性	可行	Go/Gin/Vue/Nuxt 等栈成熟稳定，PayPal 等集成有现成方案，架构清晰可落地。
经济可行性	可行	全开源技术、无授权费用，开发与部署成本可控，适合毕业设计。
操作可行性	可行	管理端与用户端界面清晰、流程符合习惯，部署与运维操作简单。
社会可行性	可行	符合教学与电商合规场景，无显著法律与伦理风险。

综合以上分析，**本跨境电商独立站系统在技术、经济、操作和社会四个维度均具备可行性**，可以进入下一阶段的需求分析与系统设计。

第三章 系统需求分析

需求分析是软件开发生命周期中的重要阶段，旨在明确系统“做什么”，为后续设计与实现提供依据。本章从用户角色、功能需求和非功能需求三个方面对本跨境电商独立站系统进行需求分析，给出功能模块划分与接口需求概要。

3.1 系统概述与需求分析目标

本系统为**跨境电商独立站**，采用前后端分离架构，由以下三部分构成：

- **后端 API 服务（server）**：基于 Go + Gin 构建，为管理后台与前台商城提供统一的 RESTful API，负责业务逻辑、数据持久化、认证与支付对接。
- **管理后台（shop_admin）**：基于 Vue 3 + Vite 构建，面向商家或运营人员，用于商品、订单、内容、站点与系统管理。
- **前台商城（zhuyi-store）**：基于 Nuxt 4 构建，面向终端消费者，用于商品浏览、购物车、下单、支付与账户管理。

需求分析的目标是：明确各用户角色及其使用场景，梳理管理端、客户端与后端的功能需求，界定性能、安全、可用性等非功能需求，为第四章系统总体设计提供输入。

图 3-1 系统顶层数据流图 如图 3-1 所示，外部实体包括管理员、前台用户与游客；系统边界内包含管理后台（shop_admin）、前台商城（zhuyi-store）与后端 API（server）；数据存储为 MySQL、Redis 与 OSS；外部系统为 PayPal。管理员通过管理后台发送商品、订单、内容等管理请求，数据经后端处理后写入 MySQL 或 Redis；前台用户与游客通过前台商城发送浏览、加购、下单等请求，数据经后端处理后读写 MySQL/Redis，支付环节与 PayPal 交互；静态资源经 OSS 存储与访问。

代码块

```
flowchart LR
    subgraph 外部实体
        A[管理员]
        B[前台用户]
        C[游客]
    end
    subgraph 系统
        D[shop_admin<br/>管理后台]
        E[zhuyi-store<br/>前台商城]
```

```
        F[server<br/>后端API]
    end
    subgraph 数据存储
        G[(MySQL)]
        H[(Redis)]
        I[OSS]
    end
    K[PayPal]
    A -->|管理请求| D
    B -->|浏览/下单/支付| E
    C -->|浏览/加购| E
    D -->|/api/manage| F
    E -->|/api/client| F
    F -->|读写| G
    F -->|Session/缓存| H
    F -->|文件| I
    F <-->|支付/退款| K
```

3.2 用户角色分析

系统涉及两类主要用户角色：**管理员**（后台使用者）与**前台用户**（商城消费者）。此外，系统需支持**游客**（未登录用户）进行浏览与加购，并通过设备指纹关联购物车与订单。

角色	说明	主要使用系统	认证方式
管理员	商家或运营人员，管理商品、订单、内容与系统	shop_admin + 后端 /api/manage	Redis Session（登录后获得 Token）
前台用户	已注册消费者，可下单、支付、管理地址等	zhuyi-store + 后端 /api/client	JWT（登录后获得 accessToken）
游客	未登录访客，可浏览、加购、按查询码查订单	zhuyi-store + 后端 /api/client	设备指纹（X-Device-Fingerprint）

约束与约定：

- 管理员仅能通过管理后台登录，使用 `/api/manage` 下接口；未携带有效 Session 的请求将被拒绝。
- 前台用户通过商城登录/注册后，访问需登录的客户端接口时需携带 JWT。
- 游客访问客户端接口时需携带设备指纹，用于购物车与订单的关联；下单时可填写收货信息，无需强制注册。

3.3 功能需求分析

功能需求按**管理端**、**客户端**与**后端**三个维度进行梳理，并与实际实现的模块对应。

3.3.1 管理端功能需求

管理端（shop_admin）面向管理员，需提供以下功能模块。

（1）仪表板。 展示站点核心运营数据：总用户数、商品数、订单数、销售额等统计；支持用户与订单增长趋势图（如 ECharts 折线图），便于运营决策。数据来源于后端统计接口或聚合查询。

（2）商品管理。 商品列表（分页、筛选、搜索）、商品新增、编辑、详情查看、软删除；商品与分类、SKU、属性、属性值、详情内容、标签等关联管理。分类管理：分类树展示、分类增删改查。商品属性与属性值：列表、创建、修改、删除。标签管理：标签列表、详情、增删改，及与商品的关联。需支持富文本编辑（如 WangEditor）与图片上传（OSS）。

（3）订单管理。 订单列表（分页、状态筛选）、订单详情查看、订单状态更新、发货（填写物流信息）、退款操作。退款列表、按订单查退款；支持调用后端 PayPal 退款接口完成退款。

（4）内容管理（CMS）。 文档/文章管理：创建、更新、列表、删除；支持归档。推荐位管理：推荐位与推荐项（推荐位内容）的增删改查，用于前台首页等区块的运营配置。

（5）站点管理。 商店设置、站点信息（名称、Logo、联系方式等）、市场/营销设置（如运费、营销开关等），供前台与后台共用。

（6）系统管理。 管理员 CRUD、列表（与部门、角色关联）；部门树与部门增删改查；角色与权限的 CRUD；前台用户列表与删除。字典属性、图标查看等辅助功能，便于后台表单与展示统一。

（7）资源与文件。 单文件/多文件上传、删除、文件信息查询；静态资源通过 `/api/oss` 映射，供商品图、文档图等使用。

以上功能均通过调用后端 `/api/manage` 下相应接口实现，需在请求中携带管理员 Token（Redis Session）。

3.3.2 客户端功能需求

客户端（zhuyi-store）面向前台用户与游客，需提供以下功能。

（1）首页。 商品列表展示、分页；可选推荐位、分类入口、营销信息等；支持 SEO 与首屏性能（如 Nuxt 预渲染）。

（2）分类与商品。 分类集合页（按分类 code 展示该分类下商品，分页）；分类树、分类详情、按 code/父级查询。商品详情页：主图/画廊、价格、SKU 选择、属性、富文本详情、规格参数；“加入购物车”（需选择 SKU、数量）；“猜你喜欢”等基于标签的推荐。

（3）购物车。 购物车列表、修改数量、删除项、小计/折扣/合计；未登录时支持游客加购（依赖设备指纹），登录后与用户绑定；提供跳转结算入口。

（4）结算与支付。 结算页：收货信息表单（姓名、邮箱、电话、国家/省/城市/邮编、街道），国家/省联动（如基于本地 address 数据）；支付方式为 PayPal。提交后创建订单，再跳转 PayPal 完成支付。订单详情页：按订单号或查询码查看订单信息、商品、金额、收货地址、状态；游客可通过查询码与设备指纹查看自己的订单。

（5）用户与账户。 登录（邮箱+密码）、注册（邮箱、用户名、密码）；登录成功后写 Cookie（如 accessToken），支持 redirect 回跳。收货地址管理：地址列表、新增、编辑、详情、删除、设默认；依赖国家/省选项与后端地址接口。

（6）内容（博客/CMS）。 文档列表（如 Footer 多列展示，链到文档详情页）；文档详情页按 code 展示标题、时间、作者、阅读数、正文（富文本）、来源等。推荐位内容用于首页等区块展示。

（7）市场与站点。 面包屑根据当前路由（分类页/商品页）动态生成（调用后端 breadcrumb 接口）；站点信息、运费等在结算等流程中使用。

以上功能通过调用后端 `/api/client` 下相应接口实现；需登录的接口携带 JWT，所有客户端请求建议携带设备指纹。

3.3.3 后端功能需求（API 与业务逻辑）

后端（server）需同时支撑管理端与客户端，并保证数据一致性与安全。按业务域划分如下。

（1）核心系统（core）。 管理员：登录（校验账号密码，写 Redis Session）、获取当前管理员信息、枚举字典；管理员 CRUD、列表（与部门、角色关联）。部门：部门树、增删改查。角色、权限：列表、详情、增删改。系统配置：站点信息、市场/营销设置（后台维护）；前台获取站点信息等。

（2）商品与店铺（sp/shop）。 商品：管理端—创建、列表、详情、修改、软删除；客户端—商品列表（分页/筛选）、商品详情。分类：管理端—分类树、增删改查；客户端—分类树、分类详情、按 code/父级查询。商品属性与属性值：管理端—列表、创建、修改、删除。标签：管理端—列表、详情、增删改；客户端—按 code 查标签、标签列表。SKU：与商品联动，多规格库存与索引。市场：客户端—站点信息、面包屑、运费等。

（3）订单与支付（sp_order / paypal）。 订单：管理端—列表、详情、更新状态、发货、退款；客户端—创建订单、按查询码查订单、订单列表。订单项、收货地址随订单读写。退款：管理端—退款列表、按订单查退款；PayPal 退款接口（后台）。PayPal：客户端—创建支付、捕获订单；后台—退款；日志表预留（paypal_order_logs、paypal_webhook_logs）。

（4）用户与端侧（mp/client）。 前台用户：管理端—用户列表、删除；客户端—注册、登录（签发 JWT）、Token 刷新/校验。用户地址：客户端—列表、新增、修改、详情、删除。购物车：客户端—列表、加购/修改/删除，支持设备指纹与用户 ID 关联。

（5）内容与运营（cms）。 文档：管理端—创建、更新、列表、删除；客户端—文档列表、按 code 查详情；支持归档。推荐位：管理端—推荐位与推荐项增删改查；客户端—推荐索引列表。

（6）资源与配置。 OSS：管理端—上传单/多文件、删除、文件信息；静态资源 `/api/oss` 映射本地目录。市场/站点配置：前后台共用配置来源。

接口与认证约束： 管理端接口统一在 `/api/manage` 下，除登录接口外均需管理员 Session。客户端接口统一在 `/api/client` 下，需设备指纹中间件；需登录的接口再校验 JWT。公开接口仅保留 `POST /api/manage/core/auth/login`（管理员登录）。

图 3-2 前台用户购物业务流程图 核心购物流程如图 3-2 所示：用户从首页或分类页浏览商品，进入商品详情后选择 SKU 与数量加入购物车（游客依赖设备指纹）；购物车页可修改数量或删除项，点击结算进入结算页；填写收货信息后提交创建订单，跳转 PayPal 完成支付；支付成功后返回订单详情页查看订单状态。

代码块

```
flowchart TD
    A[开始] --> B[浏览商品/分类]
    B --> C[商品详情]
    C --> D{已登录?}
    D -->|是| E[加购到购物车]
    D -->|否| F[设备指纹加购]
    E --> G[购物车列表]
    F --> G
    G --> H[修改数量/删除]
    H --> I[去结算]
    I --> J[填写收货信息]
    J --> K[创建订单]
    K --> L[跳转PayPal支付]
    L --> M{支付成功?}
    M -->|是| N[返回订单详情]
    M -->|否| O[结束]
    N --> O
```

3.4 非功能需求分析

非功能需求描述系统在性能、安全、可用性、可维护性等方面的要求。

（1）性能。 后端采用 Go 与 Gin，具备高并发处理能力；数据库使用 MySQL 索引与合理查询，避免 N+1；Redis 用于 Session 与热点缓存，减轻数据库压力。前端首屏：Nuxt 支持 SSR/预渲染，关键页面可预渲染以提升首屏速度；管理后台为 SPA，按需加载与分页以控制单页数据量。在毕业设计演示规模下，要求页面响应与接口响应在可接受范围内（如常规操作 2 秒内完成）。

（2）安全。 管理员密码与用户密码均需加密存储（如 bcrypt）；管理员采用 Redis Session，防止伪造；前台用户采用 JWT，设置合理过期时间与刷新机制。接口需防越权：管理端接口仅允许已登录管理员访问；客户端接口按业务区分公开（如商品列表）与需登录（如地址、下单）。支付：PayPal 对接使用官方 SDK，订单与金额由服务端校验，不信任前端传参。设备指纹仅用于关联游客购物车与订单，不替代登录认证。

（3）可用性与兼容性。 前台商城需适配常见浏览器（Chrome、Firefox、Safari、Edge）及不同分辨率（响应式布局）。管理后台主要面向桌面端。后端通过 CORS 配置支持前后端分离部署；配置（如数据库、Redis、PayPal）通过 YAML 管理，便于不同环境切换。

（4）可维护性与可扩展性。 后端采用 Handler → Service → Repository 分层与工厂模式，模块边界清晰，便于新增业务与单元测试。前端按页面与组件划分，API 封装统一（如 `app/api`、axios 封装），便于替换接口或扩展页面。数据模型与接口风格统一（如 RESTful、统一响应码与错误信息），便于联调与文档维护。

（5）可部署性。 后端可编译为单一可执行文件，依赖配置文件与 MySQL/Redis 即可运行。前端可构建为静态资源或 Node 服务，通过环境变量配置 API 基地址。满足本地与单机/云主机部署的毕业设计演示需求。

3.5 需求分析小结

维度	主要内容
用户角色	管理员（Session）、前台用户（JWT）、游客（设备指纹）；对应管理端与客户端。
管理端功能	

	仪表板、商品/分类/属性/标签、订单/退款、CMS 文档/推荐位、站点管理、系统管理、OSS。
客户端功能	首页、分类与商品、购物车、结算与 PayPal 支付、订单查询、登录注册、地址、博客/CMS、面包屑与站点信息。
后端功能	core、商品与店铺、订单与支付、用户与端侧、CMS、资源与配置；/api/manage 与 /api/client 双端接口。
非功能需求	性能、安全（密码、Session、JWT、越权与支付校验）、可用性与兼容性、可维护性与可扩展性、可部署性。

本章明确了系统用户角色、管理端与客户端的业务功能以及后端 API 与业务逻辑需求，并给出了非功能需求要点，为下一章系统总体设计提供了需求依据。

第四章 系统总体设计（概要设计）

总体设计（也称概要设计）是在需求分析的基础上，确定系统的整体架构、子系统划分、关键技术与接口规范，为详细设计与实现提供蓝图。本章从系统总体架构、前后端分离与通信方案、子系统划分与职责、认证与安全设计、数据库总体设计和接口总体设计六个方面进行描述。

4.1 系统总体架构

本系统采用**前后端分离**的 B/S 架构，由**后端 API 服务（server）**、**管理后台（shop_admin）**与**前台商城（zhuyi-store）**三个子系统构成。前端通过 HTTP/HTTPS 调用后端 RESTful API 获取数据与

执行业务，后端统一负责业务逻辑、数据持久化与第三方对接（如 PayPal、邮件）。整体架构如图 4-1 所示。

图 4-1 系统总体架构图

代码块

```
flowchart TB
    subgraph 前端
        subgraph shop_admin["shop_admin (管理后台)"]
            A1[Vue3+Vite<br/>Ant Design Vue<br/>端口:3001]
        end
        subgraph zhuyi["zhuyi-store (前台商城)"]
            A2[Nuxt 4+Nuxt UI<br/>端口:3000]
        end
    end
    subgraph server["server (后端 API)"]
        B1[Handler层]
        B2[Service层]
        B3[Repository层]
        B1 --> B2 --> B3
    end
    subgraph 存储
        C1[(MySQL)]
        C2[(Redis)]
        C3[OSS]
    end
    A1 -->|/api/manage| server
    A2 -->|/api/client| server
    B3 --> C1
    B3 --> C2
    B3 --> C3
    server -.->|PayPal| D[PayPal]
```

（1）总体拓扑。 管理后台（Vue 3 + Vite，默认端口 3001）与前台商城（Nuxt 4，默认端口 3000）作为两个独立的前端应用，分别部署；后端 API（Go + Gin，默认端口 8080）单独部署，对外提供统一入口 `/api`。管理后台仅调用 `/api/manage` 下接口，前台商城仅调用 `/api/client` 下接口，实现管理端与客户端在接口与认证上的隔离。

（2）后端内部架构。 后端采用**分层架构**：请求经 Gin 路由进入 **Handler 层**（HTTP 处理），Handler 调用 **Service 层**（业务逻辑），Service 调用 **Repository 层**（数据访问）；Repository 通过 GORM 操作 MySQL，会话与缓存通过 Redis 实现。各层通过 **Factory** 统一获取 Repository 与 Service 实例，避免直接 new 带来的耦合，便于扩展与测试。配置通过 YAML 文件加载（Viper），包含数据库、Redis、邮件、PayPal、前端 URL 等。

（3）数据与外部依赖。 持久化数据存放在 MySQL；管理员会话与可选缓存存放在 Redis；静态文件（如 OSS 上传）映射到本地目录并通过 `/api/oss` 提供访问。支付对接 PayPal（创建订单、捕获订单、退款），邮件通过 SMTP（如 gomail）发送。

综上，系统总体架构为**前后端分离 + 后端分层（Handler → Service → Repository）+ 双端 API 分组（/api/manage、/api/client）**，职责清晰，便于部署与扩展。

4.2 前后端分离与通信方案

（1）分离方式。 前端与后端在**开发、部署、运行**上完全分离：前端使用 Vue/Nuxt 技术栈，构建为静态资源或 Node 服务；后端使用 Go 技术栈，编译为独立可执行文件。两者通过 **RESTful API** 通信，数据格式为 **JSON**。前端不直接访问数据库，所有业务与数据访问均经后端 API 完成。

（2）通信协议与格式。 采用 HTTP/HTTPS，请求方法遵循 REST 习惯（如 GET 查询、POST 创建、PUT/PATCH 更新、DELETE 删除）。请求头需包含 `Content-Type: application/json`（POST/PUT 等）；跨域由后端 **CORS 中间件** 统一放行前端域名，便于开发与生产环境下的前后端分离部署。

（3）响应规范。 后端统一封装响应结构，例如：`{ "code": 0, "message": "ok", "result": { ... } }`，其中 `code` 为业务状态码（如 0 表示成功），`result` 为业务数据。前端（如 axios 封装）根据 `code` 判断成功与否，并解析 `result` 用于页面展示；错误时可根据 `message` 或错误码做统一提示。

（4）API 基地址与环境。 管理后台通过环境变量（如 `VITE_API_BASE_URL`）配置后端基地址，实际请求为 `VITE_API_BASE_URL + /api/manage/...`。前台商城通过运行时配置（如 Nuxt 的 `runtimeConfig.public.apiUrl`）配置客户端 API 基地址，实际请求为 `apiUrl`（通常为 `http(s)://域名:8080/api/client`）。开发与生产可通过不同环境变量切换，无需改代码。

4.3 子系统划分与职责

系统划分为三个子系统，职责如下。

子系统	技术栈	职责简述
server	Go + Gin + GORM + MySQL + Redis	提供 RESTful API；实现业务逻辑与数据访问；管理员 Session、用户 JWT、设备指纹；PayPal、邮件、OSS。
shop_admin	Vue 3 + Vite + Ant Design Vue + Pinia	管理后台界面；仪表板、商品/订单/CMS/站点/系统管理；调用 /api/manage，携带管理员 Token。
zhuyi-store	Nuxt 4 + Vue 3 + Nuxt UI + Tailwind	前台商城界面；首页、分类、商品、购物车、结算、支付、订单、用户、博客；调用 /api/client，携带设备指纹与可选 JWT。

（1）server（后端 API）。 作为唯一的数据与业务中心，负责：用户与权限认证（管理员 Redis Session、前台 JWT、设备指纹中间件）；核心、商品、订单、用户、CMS、资源等业务模块的接口实现；与 MySQL、Redis、PayPal、SMTP 的集成。不包含任何前端页面，仅输出 JSON。

（2）shop_admin（管理后台）。 面向管理员，负责：登录页与后台布局；仪表板、商品管理、订单管理、内容管理、站点管理、系统管理、OSS 上传等页面的展示与交互；将用户操作转化为对 `/api/manage` 的请求，并处理响应与错误。不直接访问数据库。

（3）zhuyi-store（前台商城）。 面向终端用户与游客，负责：首页、分类、商品详情、购物车、结算、订单详情、登录注册、地址、博客等页面的展示与交互；将用户操作转化为对 `/api/client` 的请求，携带设备指纹与登录态（JWT）；PayPal 支付跳转与回调处理在前后端配合下完成。不直接访问数据库。

三者通过**接口契约**（URL、方法、参数与响应格式）协作，实现“前端负责展示与交互、后端负责业务与数据”的职责分离。

4.4 认证与安全设计

（1）管理员认证。 管理员登录接口为 `POST /api/manage/core/auth/login` （公开）。登录成功后，后端在 Redis 中建立 Session，并将 Session ID 通过响应返回（如 token 字段）。前端将 token 保存在 Cookie 或本地存储，后续请求通过 Header（如 `Authorization: Bearer <token>` ）、Query（ `token=` ）或 Cookie（如 `YEX_AUTH` ）携带。后端在 `/api/manage` 路由组上挂载**管理员认证中间件**，从请求中解析 token，用 Redis 校验 Session；通过则把管理员信息写入上下文（如 user、userID），否则返回 401。这样，除登录接口外，所有管理端接口均需有效 Session。管理员登录流程如图 4-3 所示。

代码块

```
flowchart TD
    A[管理员输入账号密码] --> B[提交登录请求]
    B --> C[后端校验账号密码]
    C --> D{校验通过?}
    D -->|否| E[返回错误信息]
    D -->|是| F[Redis写入Session]
    F --> G[返回token]
    G --> H[前端保存token]
    H --> I[后续请求携带token]
    I --> J[中间件校验Session]
    J --> K{有效?}
    K -->|是| L[访问管理接口]
    K -->|否| M[401未授权]
```

（2）前台用户认证。 前台用户通过注册/登录接口获取 JWT。后端签发 JWT（包含 userID、email 等），前端将 JWT 存入 Cookie（如 accessToken）或本地存储。访问需登录的客户端接口时，在 Header 中携带 `Authorization: Bearer <accessToken>` 。后端在 `/api/client` 路由组上挂载**可选认证中间件**：解析 JWT，若有效则将 userID、email 等写入上下文；若无效或未携带，则不写入用户信息，接口内根据业务判断是否要求登录（如下单可支持游客+收货信息，地址列表则需登录）。Token 刷新/校验接口用于延长会话或检查登录态。

（3）设备指纹。 客户端请求统一经过**设备指纹中间件**，从 Header（`X-Device-Fingerprint`）、Query 或 Cookie 中读取设备指纹并写入上下文。用于：游客购物车与订单的关联（同一设备指纹视为同一“游客”）；按查询码查订单时结合设备指纹校验归属。设备指纹由前端生成（如基于浏览器特征或 Cookie device_id），不替代登录认证，仅作辅助标识。

（4）安全要点。 密码存储：管理员与用户密码均不可明文存储，采用 bcrypt 等单向加密。接口越权：管理端接口仅允许已登录管理员；客户端接口按业务区分公开（如商品列表）与需登录（如地址、下单），服务端根据上下文 userID 做数据隔离。支付安全：PayPal 订单与金额由后端生成与校验，不信任前端传参；退款仅由管理端发起并调用后端 PayPal 退款接口。CORS：后端 CORS 中间件按需配置允许的前端来源，避免任意站点调用 API。

图 4-2 后端请求处理与认证流程图 如图 4-2 所示，请求进入 Gin 路由后先经 CORS 中间件，管理端请求经管理员认证中间件（校验 Redis Session），客户端请求经设备指纹中间件与可选 JWT 中间件，随后进入对应 Handler；Handler 调用 Service，Service 调用 Repository 访问 MySQL/Redis，最终返回 JSON 响应。

代码块

```
flowchart TD
    A[HTTP请求] --> B[CORS中间件]
    B --> C{路由组?}
    C -->|/api/manage| D[管理员认证中间件]
    C -->|/api/client| E[设备指纹中间件]
    D --> F{Session有效?}
    F -->|是| G[Handler]
    F -->|否| H[401]
    E --> I[可选JWT中间件]
    I --> G
    G --> J[Service]
    J --> K[Repository]
    K --> L[(MySQL/Redis)]
    L --> K
    K --> J
    J --> G
    G --> M[统一JSON响应]
```

4.5 数据库总体设计

（1）选型与职责。 关系型数据使用 **MySQL**，通过 **GORM** 进行 ORM 映射与迁移；会话与缓存使用 **Redis**，不在此处展开表结构。数据库按**业务域**划分逻辑模块，与后端 models/repository 目录对应。

（2）核心域（core）。 管理员（core_admin）、部门（core_dept）、角色（core_role）、权限（core_permission）、系统配置（core_config）等表，支撑后台登录、权限与站点配置。

（3）商品与店铺域（sp/shop）。 商品（sp_product）、分类（sp_category）、商品属性与属性值（sp_prod_attributes、sp_prod_attributes_value）、SKU（sp_sku、sp_sku_index）、商品内容与属性（sp_product_content、sp_product_property）、标签（shop_tag、shop_tag_index、shop_tag_mate）等表，支撑商品、分类、规格与标签管理。

（4）订单与支付域（sp_order / paypal）。 订单（sp_order）、订单项（sp_order_item）、收货地址（sp_order_receive_address）、退款（sp_order_refund）、PayPal 日志（paypal_order_logs、paypal_webhook_logs，预留）等表，支撑下单、发货、退款与支付流水。

（5）用户与端侧域（mp）。 前台用户（mp_user）、用户 Token（mp_user_token）、用户地址（sp_user_address）、购物车（sp_user_cart）等表，支撑注册登录、地址与购物车。

（6）内容与运营域（cms）。 文档（cms_document）、文档归档（cms_document_archive）、推荐位（cms_recommend）、推荐项（cms_recommend_index）等表，支撑 CMS 文档与推荐位。

（7）设计原则。 表名与字段命名统一风格（如小写+下划线）；主键一般采用自增 ID 或 UUID；外键关系在模型中通过 GORM 关联表达，便于关联查询；敏感字段（如密码）不落库明文；关键业务表预留创建/更新时间等审计字段。

表 4-1 至表 4-6 给出部分核心数据表结构，与 README 及项目实现对应。

表 4-1 core_admin（管理员表）

字段名	类型	说明
id	bigint	主键，自增
username	varchar(64)	用户名

password	varchar(255)	密码（密文）
dept_id	bigint	部门 ID
role_id	bigint	角色 ID
status	tinyint	状态
created_at	datetime	创建时间
updated_at	datetime	更新时间

表 4-2 sp_product（商品表）

字段名	类型	说明
id	bigint	主键，自增
title	varchar(255)	商品标题
category_id	bigint	分类 ID
main_image	varchar(500)	主图 URL
price	decimal(10,2)	价格
status	tinyint	状态（上架/下架）
deleted_at	datetime	软删除时间
created_at	datetime	创建时间
updated_at	datetime	更新时间

表 4-3 sp_order（订单表）

字段名	类型	说明
id	bigint	主键，自增
order_no	varchar(64)	订单号

query_code	varchar(32)	查询码（游客）
user_id	bigint	用户 ID（可为空）
device_fingerprint	varchar(64)	设备指纹
total_amount	decimal(10,2)	订单总金额
status	tinyint	订单状态
created_at	datetime	创建时间
updated_at	datetime	更新时间

表 4-4 sp_order_item（订单项表）

字段名	类型	说明
id	bigint	主键，自增
order_id	bigint	订单 ID
product_id	bigint	商品 ID
sku_id	bigint	SKU ID
quantity	int	数量
price	decimal(10,2)	单价

表 4-5 mp_user（前台用户表）

字段名	类型	说明
id	bigint	主键，自增
email	varchar(128)	邮箱
username	varchar(64)	用户名

password	varchar(255)	密码（密文）
status	tinyint	状态
created_at	datetime	创建时间
updated_at	datetime	更新时间

表 4-6 cms_document（文档表）

字段名	类型	说明
id	bigint	主键，自增
code	varchar(64)	文档编码
title	varchar(255)	标题
content	longtext	正文（富文本）
status	tinyint	状态
created_at	datetime	创建时间
updated_at	datetime	更新时间

4.6 接口总体设计

（1）接口分组与认证。 所有 API 统一挂在 `/api` 下，分为三组：**公开**（仅管理员登录）、**管理端**（`/api/manage/*`，需管理员 Session）、**客户端**（`/api/client/*`，需设备指纹，部分需 JWT）。分组与第三章需求、README 中路由归纳一致。

（2）管理端接口概要。 认证：`POST /api/manage/core/auth/login`。核心：core/auth（当前管理员、枚举字典）、core/admin、core/dept、core/role、core/permission、core/oss。商品与店铺：shop/product、shop/category、shop/prodAttributes、shop/prodAttributesValue、shop/tag、shop/marketSetting。内容：shop/document、shop/recommend、shop/recommendIndex。订单与支付：shop/order（列表、详情、状态、发货、退款）、shop/refund、payment/paypal（退款）。用户：mp/user（列表、删除）。请求方法以 REST 为准

（如 GET 列表/详情、POST 创建、PUT/PATCH 更新、DELETE 删除）；具体路径与参数在第五章及接口文档中细化。

（3）客户端接口概要。 内容：shop/document（列表、详情）、shop/recommendIndex。商品与店铺：shop/product（列表、详情）、shop/category（树、详情、按 code/父级）、shop/market（站点信息、面包屑、运费）、shop/tag（列表、按 code）。购物车：shop/userCart（列表、加购/改数量/删除）。用户与订单：shop/userAuth（登录、注册）、shop/order（创建订单、按查询码查订单、订单列表）、shop/userAddress（CRUD）。支付：payment（PayPal 创建订单、捕获订单）。请求方法同样遵循 REST；需登录的接口在业务层校验上下文中是否存在 userID。

（4）统一约定。 响应体统一为 JSON；成功时 `code` 为 0，业务数据在 `result` 中；失败时 `code` 非 0，`message` 为错误说明。列表接口支持分页参数（如 page、pageSize）；筛选与排序参数按业务约定（如订单状态、商品分类）。这样，前端与后端对“成功/失败”和“数据位置”有统一理解，便于联调与错误处理。

4.7 总体设计小结

维度	设计要点
总体架构	前后端分离 B/S；server + shop_admin + zhuyi-store；后端 Handler→Service→Repository 分层 + Factory。
前后端通信	RESTful API、JSON；CORS；统一响应结构（code、message、result）；API 基地址通过环境配置。
子系统	server（业务与数据）、shop_admin（管理端 UI）、zhuyi-store（客户端 UI）；职责清晰、仅通过 API 协作。
认证与安全	管理员 Redis Session；前台 JWT + 可选认证中间件；设备

	指纹；密码加密、越权防护、PayPal 服务端校验。
数据库	MySQL 按 core、sp/shop、订单/支付、mp、cms 等域划分；GORM 映射；Redis 会话与缓存。
接口	/api 下公开、/api/manage、/api/client 三组；管理端与客户端接口列表与认证方式明确；统一响应约定。

本章给出了系统总体架构、前后端分离与通信方案、子系统划分、认证与安全、数据库与接口的概要设计，为第五、六、七章的后端、管理后台与前台商城的详细设计与实现提供了依据。

第五章 详细设计

详细设计是在总体设计的基础上，对系统各组成部分进行细化，明确模块职责、类/接口定义、数据流与关键实现思路，为编码实现提供可直接落地的设计说明。本章以**后端服务（server）**为核心，详述分层架构、路由与中间件、核心业务模块的设计，以及关键接口与数据模型；**管理后台（shop_admin）**与前台商城（zhuyi-store）**的详细设计，包括技术选型、模块/页面结构、主要组件与流程设计、API 对接方式，为第六章系统实现提供完整设计依据。

5.1 详细设计概述

本系统后端采用 **Go + Gin** 实现，目录按职责划分为：**cmd**（入口）、**config**（配置）、**handlers**（HTTP 处理）、**middleware**（中间件）、**models**（数据模型）、**repository**（数据访问）、**router**（路由）、**service**（业务逻辑）、**utils**（工具）。设计原则包括：**分层清晰**（Handler → Service → Repository）、**依赖注入**（通过 Factory 获取 Repository 与 Service）、**接口与认证分离**（/api/manage 与 /api/client 分组）、**统一响应与错误处理**。以下各节按分层与模块展开详细设计。

5.2 后端分层架构详细设计

5.2.1 整体分层与调用关系

请求自 Gin 路由进入后，依次经过**中间件**（CORS、认证、设备指纹等）→ **Handler** → **Service** → **Repository** → **MySQL/Redis**。Handler 不直接访问数据库，仅负责解析请求、调用 Service、封装响应；Service 负责业务规则与事务边界；Repository 负责 CRUD 与查询封装。各层通过**接口或工厂**获取下一层依赖，便于单元测试与替换实现。分层调用关系如图 5-1 所示。

图 5-1 后端分层架构与调用关系图

代码块

```
flowchart TB
    subgraph 请求入口
        A[HTTP 请求]
    end
    subgraph 中间件
        B[CORS]
        C[管理员认证/设备指纹/JWT]
    end
    subgraph Handler层
        D[handlers/admin]
        E[handlers/client]
    end
    subgraph Service层
        F[AuthService]
        G[ProductService]
        H[OrderService]
        I[其他Service]
    end
    subgraph Repository层
        J[AdminRepo]
        K[ProductRepo]
        L[OrderRepo]
        M[其他Repo]
    end
    subgraph 存储
        N[(MySQL)]
        O[(Redis)]
    end
```

```
    A --> B --> C --> D
    C --> E
    D --> F
    E --> G
    E --> H
    F --> J
    G --> K
    H --> L
    J --> N
    K --> N
    L --> N
    F --> O
```

表 5-1 各层职责对照表

层级	职责概要	典型方法/组件
Handler 层	解析请求、参数校验、调用 Service、封装响应	GetProductList、CreateOrder
Service 层	业务规则、事务边界、多表协调、外部调用	Login、CreateProduct、CapturePayPal
Repository 层	CRUD、复杂查询、关联加载	GetByID、List、Create、Update
Factory	统一创建 Repository 与 Service	GetProductRepo、GetOrderService

5.2.2 Handler 层（HTTP 处理层）

职责：解析 HTTP 请求（Query、Body、Header）、校验参数格式、从上下文读取认证信息（如 userID、token、设备指纹）、调用对应 Service 方法、将 Service 返回结果封装为统一 JSON 响应（code、message、result）；错误时返回非 0 的 code 与 message。

组织方式：按端划分目录，**handlers/admin** 对应管理端接口（商品、订单、CMS、权限等），**handlers/client** 对应客户端接口（商城、用户、支付等）；**handlers/constant** 存放业务常量（如

状态码、枚举）。每个 Handler 函数接收 `*gin.Context`，从上下文获取已注入的 Service（或通过全局/请求级 Factory 获取），调用 Service 后使用 **utils 中的统一响应方法**（如 `utils.Success(c, result)`、`utils.Fail(c, code, message)`）写回响应。

典型流程：解析请求体 → 参数校验（必填、格式）→ 调用 `service.XXX()` → 根据返回错误或数据调用 `utils.Success` 或 `utils.Fail`。不在此层写业务逻辑与 SQL。

5.2.3 Service 层（业务逻辑层）

职责：实现业务规则、事务边界、多表协调、与外部服务对接（如 PayPal、邮件）。不直接依赖 `*gin.Context`，仅依赖 Repository 与必要的配置/工具（如 JWT 签发、密码校验）。

组织方式：**service** 目录下按业务域划分子包或文件，如 core（管理员、部门、角色、权限、配置）、shop（商品、分类、属性、标签、市场）、order（订单、退款）、mp（用户、地址、购物车）、cms（文档、推荐）、payment（PayPal）、oss（文件上传）。各 Service 通过构造函数或 **ServiceFactory** 注入所需的 Repository；Service 间若有调用，也通过 Factory 获取，避免循环依赖。

典型流程：接收 DTO 或领域参数 → 校验业务规则（如库存、订单状态）→ 调用一个或多个 Repository 完成读写 → 必要时调用 PayPal/邮件等 → 返回结果或业务错误。涉及多表写操作时，在同一 Service 方法内开启事务（如 `db.Transaction`），由 Repository 使用同一 tx 执行。

5.2.4 Repository 层（数据访问层）

职责：封装对 MySQL 的 CRUD 与复杂查询，不包含业务判断。方法接收模型 ID 或查询条件，返回实体、列表或影响行数；复杂查询可返回 DTO 或 map，由 Service 组装。

组织方式：**repository** 目录下按模型或聚合根划分子包，与 **models** 目录对应（如 core、sp、shop、mp、cms、mypaypal）。各 Repository 通过 **RepositoryFactory** 统一创建，避免在 Handler/Service 中直接 new 具体实现；Factory 在 **cmd/main.go** 或 **config** 初始化时构建，并注入到 Router 或 Handler 的依赖中。

典型方法：`GetByID`、`List`（分页、筛选）、`Create`、`Update`、`Delete`（或软删除）；按业务需要的 `FindByCode`、`FindByOrderID` 等。使用 GORM 的 `db.Where`、

`Preload` 做关联加载，避免 N+1 查询。

5.2.5 Factory 与依赖注入

RepositoryFactory：提供各模块 Repository 的获取方法（如 `GetAdminRepo()`、`GetProductRepo()`），内部持有 `*gorm.DB`，在创建 Repository 时传入。若 Repository 为无状态，可复用单例；若有请求级作用域，由调用方或中间件按需创建。

ServiceFactory：提供各业务 Service 的获取方法（如 `GetAuthService()`、`GetProductService()`），内部依赖 RepositoryFactory 或具体 Repository，在创建 Service 时注入。同样可单例或按请求创建。

注入时机：在 **router** 注册路由时，将 Factory 或已创建的 Handler 传入路由组；Handler 在初始化时接收 Service（或 Factory），从而在请求处理中调用对应 Service。这样，分层与依赖方向清晰：Handler → Service → Repository → DB。

5.3 路由与中间件详细设计

5.3.1 路由分组与挂载

入口：**router/router.go** 中创建 Gin 引擎，挂载全局 **CORS 中间件**，然后将 `/api` 作为路由组根。

分组结构：

- **公开**：`POST /api/manage/core/auth/login`（管理员登录），不经过管理员认证中间件
- **管理端**：`/api/manage` 为前缀的路由组，挂载**管理员认证中间件**；其下按业务挂子组，如 `core/auth`、`core/admin`、`core/dept`、`core/role`、`core/permission`、`core/oss`、`shop/product`、`shop/category`、`shop/prodAttributes`、`shop/prodAttributesValue`、`shop/tag`、`shop/marketSetting`、`shop/document`、`shop/recommend`、`shop/recommendIndex`、`shop/order`、`shop/refund`、`payment/paypal`、`mp/user` 等，每个子组绑定对应 Handler。

- **客户端**：`/api/client` 为前缀的路由组，先挂载**设备指纹中间件**，再挂载**可选认证中间件**（解析 JWT）；其下挂子组，如 `shop/document`、`shop/product`、`shop/category`、`shop/market`、`shop/tag`、`shop/recommendIndex`、`shop/userCart`、`shop/userAuth`、`shop/order`、`shop/userAddress`、`payment` 等，绑定对应 Handler。

静态资源：`/api/oss` 映射到本地目录（如 `./oss`），用于文件访问；上传与删除走管理端 OSS 接口。路由分组与中间件挂载关系如表 5-2 所示。

表 5-2 路由分组与中间件对照表

路由组	前缀	中间件	说明
公开	/api/manage/core/auth/login	无	仅管理员登录
管理端	/api/manage	管理员认证（Redis Session）	core、shop、payment、mp 等
客户端	/api/client	设备指纹 + 可选 JWT	shop、payment 等
静态资源	/api/oss	无	本地目录映射

5.3.2 中间件详细设计

CORS 中间件：在响应头中设置 `Access-Control-Allow-Origin`（可为配置中的前端 URL 或 `*`）、`Access-Control-Allow-Methods`、`Access-Control-Allow-Headers`、`Access-Control-Allow-Credentials` 等，支持前后端分离下的跨域请求。OPTIONS 预检请求直接返回 200。

管理员认证中间件：从 `c.GetHeader("Authorization")`、`c.Query("token")`、`c.Cookie("YEX_AUTH")` 等位置解析 token；使用 **utils.GetSession(token)** 查询 Redis，若 Session 存在则取出管理员信息（如 user、userID），并写入 `c.Set("user", ...)`、`c.Set("userID", ...)`、`c.Set("token", token)`，然后 `c.Next()`；若不存在则 `c.Abort()` 并返回 401 及统一错误格式。

设备指纹中间件：从 `c.GetHeader("X-Device-Fingerprint")`、Query 或 Cookie 读取设备指纹，若不存在可生成默认值（如 UUID）；写入 `c.Set("deviceFingerprint", fingerprint)`，然后 `c.Next()`。后续 Handler/Service 从上下文读取，用于购物车与订单关联。

可选认证中间件（客户端）：从 Header 的 `Authorization: Bearer <accessToken>` 解析 JWT；若 token 有效则解析出 userID、email 等，写入 `c.Set("userID", ...)`、`c.Set("email", ...)`、`c.Set("token", accessToken)`；若无效或未携带则不写入，不 Abort，交由具体接口判断是否必须登录。然后 `c.Next()`。

5.4 核心业务模块详细设计

本节按业务域简述各模块的职责、主要 Service 方法及与 Repository/模型的对应关系，便于实现时按图索骥。核心业务模块划分及与数据模型的对应关系如表 5-3 所示。

表 5-3 核心业务模块与数据模型对照表

业务域	主要 Service	主要 Repository	数据模型（表）
核心系统 core	AuthService, AdminService	AdminRepo, DeptRepo, RoleRepo	core_admin, core_dept, core_role, core_permission, core_config
商品与店铺 sp/shop	ProductService, CategoryService	ProductRepo, CategoryRepo, SkuRepo	sp_product, sp_category, sp_sku, shop_tag 等
订单与支付	OrderService, RefundService	OrderRepo, OrderItemRepo	sp_order, sp_order_item, sp_order_refund
用户与端侧 mp	UserService, AddressService	UserRepo, AddressRepo, CartRepo	mp_user, sp_user_address, sp_user_cart
内容 CMS			

	DocumentService, RecommendService	DocumentRepo, RecommendRepo	cms_document, cms_recommend, cms_recommend_index
资源 OSS	OssService	文件系统	本地目录 ./oss

5.4.1 核心系统（core）

管理员（core_admin）：登录时校验账号密码（与数据库密文比对），通过则在 Redis 写入 Session（key 含 token，value 含 adminID 等），并返回 token；获取当前管理员信息、枚举字典均从上下文或 Session 读取。管理员 CRUD、列表由 AdminService 实现，依赖 AdminRepository、DeptRepository、RoleRepository 等；列表支持与部门、角色关联展示。

部门（core_dept）：部门树通过 Repository 递归或一次查询+内存组装实现；增删改查为常规 CRUD，需校验父子关系（如禁止将父部门设为子部门的子节点）。

角色（core_role）、权限（core_permission）：CRUD 与列表、详情；角色与权限的关联（多对多）通过中间表或 GORM 关联维护，按需在 Service 层做绑定/解绑。

系统配置（core_config）：按 key 或类型读写配置项（如站点信息、市场/营销开关）；管理端写，客户端只读（如站点信息、运费）。可用单表 key-value 或按业务拆表，由 ConfigRepository 封装。

模型：core_admin、core_dept、core_role、core_permission、core_config 等，位于 **models/core**。

5.4.2 商品与店铺（sp / shop）

商品（sp_product）：管理端—创建时写入主表及关联（分类、SKU、属性值、详情内容、标签等），由 ProductService 协调多个 Repository（Product、Category、Sku、ProdAttributesValue、ProductContent、Tag 等）；列表支持分页、筛选、软删除过滤；详情带 Preload 加载关联。客户端—列表仅查上架、未删除，支持分页与分类/标签筛选；详情同样带关联，供前端展示与加购。

分类（sp_category）：树形结构（parent_id），管理端 CRUD 时校验 code 唯一与层级；客户端提供树、按 code/父级查询，用于导航与面包屑。

商品属性与属性值（sp_prod_attributes、sp_prod_attributes_value）：管理端对属性和属性值做 CRUD；与商品的关系在商品编辑时一并维护（如多对多或 JSON 存储）。

标签（shop_tag）：管理端 CRUD；与商品的关联通过 shop_tag_index、shop_tag_mate 等表；客户端按 code 查标签、标签列表及关联商品。

SKU（sp_sku、sp_sku_index）：由商品创建/编辑时一并生成，表示多规格与库存；加购、下单时校验 sku_id 与库存，扣减在订单支付/发货流程中由 OrderService 协调。

市场/站点（market）：客户端接口提供站点信息、面包屑（根据当前分类/商品 ID 查路径）、运费等，数据来源于 core_config 或专用配置表，由 MarketService 封装。

模型：sp_product、sp_category、sp_prod_attributes、sp_prod_attributes_value、sp_sku、sp_sku_index、sp_product_content、sp_product_property、shop_tag、shop_tag_index、shop_tag_mate 等，位于 **models/sp**、**models/shop**。

5.4.3 订单与支付（sp_order / paypal）

订单（sp_order）：创建订单由 Client 端 OrderService 实现：校验购物车或传入商品列表、收货地址、计算金额、扣减库存（可选）、生成订单与订单项、写入 sp_order、sp_order_item、sp_order_receive_address；生成订单查询码供游客查询。管理端：列表（分页、状态筛选）、详情、更新状态、发货（写物流信息）、退款（创建退款单并调 PayPal 退款）。状态流转需符合业务规则（如已支付才能发货、已发货才能部分退款）。

退款（sp_order_refund）：管理端列表、按订单查退款；退款时调用 PayPal 退款接口，并更新退款单状态与订单状态。

PayPal：客户端—创建支付（PayPal 创建订单，返回 approval_url）、捕获订单（用户支付成功后回调，后端 capture）；管理端—退款接口。PayPal 订单 ID 与本地订单 ID 的关联、金额校验均在 Service 层完成；日志写入 paypal_order_logs、paypal_webhook_logs（预留）。

模型：sp_order、sp_order_item、sp_order_receive_address、sp_order_refund、paypal_order_logs、paypal_webhook_logs，位于 **models/sp**、**models/mypaypal**。

5.4.4 用户与端侧（mp / client）

前台用户（mp_user）：注册时校验邮箱唯一、写 mp_user；登录时校验密码、签发 JWT（存 mp_user_token 可选）、返回 accessToken；Token 刷新/校验通过 JWT 验证与续期。管理端仅提供用户列表、删除。

用户地址（sp_user_address）：客户端 CRUD；列表与详情需按上下文 userID 过滤，防止越权。

购物车（sp_user_cart）：加购/改数量/删除时，若上下文有 userID 则按 userID 关联，否则按设备指纹关联（同一设备指纹视为同一“游客”）；列表接口根据 userID 或 deviceFingerprint 查询，返回当前用户/游客的购物车项。

模型：mp_user、mp_user_token、sp_user_address、sp_user_cart，位于 **models/mp**、**models/sp**。

5.4.5 内容与运营（cms）

文档（cms_document）：管理端 CRUD；支持归档（cms_document_archive）。客户端列表（分页、可选分类）、按 code 查详情，用于博客/帮助中心。

推荐位（cms_recommend、cms_recommend_index）：管理端对推荐位与推荐项（如商品、文档）增删改查；客户端按推荐位 code 返回推荐索引列表，供首页等区块展示。

模型：cms_document、cms_document_archive、cms_recommend、cms_recommend_index，位于 **models/cms**。

5.4.6 资源与配置（OSS / 市场）

OSS：管理端上传单文件/多文件时，将文件写入本地目录（如 `./oss/yyyy/mm/dd/uuid.ext`），并将访问路径（如 `/api/oss/...`）或相对路径存入业务表或返回给前端；删除为物理删除或标记删除；文件信息接口根据 path 返回元信息。静态访问通过 Gin 静态路由 `/api/oss` 映射到本地目录。

市场/站点配置：与 core_config 或专用表一致，由 MarketService 统一提供给管理端与客户端。

5.5 关键接口与数据模型

5.5.1 统一响应与错误码

成功响应：`{ "code": 0, "message": "ok", "result": { ... } }`，其中 result 为业务数据（对象或数组）。**失败响应**：`{ "code": nonZero, "message": "错误说明" }`。业务错误码可在 **handlers/constant** 中定义（如 400 参数错误、401 未登录、403 无权限、404 资源不存在、500 服务器错误）。Handler 层统一使用 utils.Success / utils.Fail，避免各处手写 JSON。

5.5.2 分页与列表约定

列表接口统一接受 `page`、`pageSize`（或 limit）；返回 result 中包含 `list`（当前页数据）与 `total`（总条数），便于前端分页与总数展示。筛选参数按业务约定，如订单状态 status、商品分类 categoryId、时间范围等，由 Handler 解析后传入 Service。主要接口响应格式如表 5-4 所示。

表 5-4 统一响应格式与分页约定

场景	请求参数	响应 result 结构
成功（单条）	-	`{ "id": 1, ... }`
成功（列表）	page, pageSize	`{ "list": [...], "total": 100 }`
失败	-	响应体 `code` 非 0，`message` 为错误说明

5.5.3 关键数据模型要点

core_admin：id、username、password（密文）、部门/角色 ID、状态、创建/更新时间等。**sp_product**：id、title、category_id、主图、价格、库存汇总、状态（上架/下架）、软删除标记、创建/更新时间等；与 sp_sku、sp_category、shop_tag 等通过外键或关联表连接。**sp_order**：id、订单号、查询码、user_id（可为空）、设备指纹、总金额、状态（待支付/已支付/已发货/已退款等）、收货地址快照、创建/更新时间等；sp_order_item 存订单项（商品、SKU、数量、单价）。**mp_user**：id、email、username、password（密文）、状态等；**mp_user_token** 可选存 JWT 或 refresh token。**cms_document**：id、code、title、content（富文本）、分类、状态、归档标记等。各表主键一般为自增 ID 或 UUID；外键在 GORM 中用 `ForeignKey` 或手动维护；创建/更新时间为 GORM 的 CreatedAt/UpdatedAt。

5.6 管理后台详细设计（shop_admin）

管理后台面向商家或运营人员，负责商品、订单、内容、站点与系统等模块的维护与配置。本节从技术选型、模块划分、主要页面与组件设计、API 对接方式四个方面进行详细设计。

5.6.1 技术选型与目录结构设计

技术选型：采用 **Vue 3 + TypeScript + Vite** 作为基础框架，**Ant Design Vue 4.x** 作为 UI 组件库，**Pinia** 作为状态管理，**Axios** 作为 HTTP 客户端，**ECharts** 用于仪表板图表，**WangEditor** 用于富文本编辑（商品详情、CMS 文档）。构建工具为 Vite，默认端口 3001；API 基地址通过环境变量 **VITE_API_BASE_URL** 配置（如 `http://localhost:8080/api`），实际请求路径为 `VITE_API_BASE_URL + /manage/...`，与后端 `/api/manage` 对应。

目录结构设计：**src/api** 存放接口封装，按业务模块分文件（如 auth、product、category、order、cms、site、system、oss），每个模块导出对应接口方法（如 getCurrentUser、getProductList、createProduct）；**src/components** 存放通用组件（如表单、表格、上传、富文本包装）；**src/views** 存放页面视图，按功能模块分子目录（如 Dashboard、Product、Order、Cms、Site、System），每个页面对应一个或多个路由；**src/router** 配置路由（登录页单独、后台布局与子路由、路由守卫校验登录态）；**src/store** 使用 Pinia 存储用户信息、Token、侧边栏状态等；**src/utils** 存放请求封装、常量、工具函数。请求封装需在 axios 实例上配置 baseURL、请求拦截器（从 Cookie 或 localStorage 读取 token 并写入 Header 的 Authorization 或 Query 的 token）、响应拦截器（根据 code 判断成功与否，401 时跳转登录）。

5.6.2 模块划分与主要页面设计

模块划分：按功能划分为**仪表板**、**商品管理**、**订单管理**、**内容管理（CMS）**、**站点管理**、**系统管理**、**资源（OSS）**。每个模块对应一个或多个菜单项与路由，页面间通过路由参数或 Pinia 共享必要状态。管理后台模块与页面、主要接口的对应关系如表 5-5 所示。

表 5-5 管理后台模块与页面/接口对照表

模块	页面/功能	主要 API 路径
仪表板	Dashboard 统计与趋势图	统计接口、趋势数据
商品管理	商品列表、新增/编辑、分类、属性、标签	shop/product, shop/category, shop/prodAttributes, shop/tag
订单管理	订单列表、详情、发货、退款	shop/order, shop/refund, payment/paypal
内容管理	文档列表、推荐位与推荐项	shop/document, shop/recommend, shop/recommendIndex
站点管理	站点信息、市场设置	shop/marketSetting
系统管理	管理员、部门、角色、权限、用户	core/admin, core/dept, core/role, core/permission, mp/user
资源	文件上传、删除	core/oss

图 5-2 管理后台布局与模块结构图

代码块

```
flowchart TB
    subgraph 布局
```

```
        A[登录页]
        B[后台布局]
    end
    subgraph 左侧菜单
        C1[仪表板]
        C2[商品管理]
        C3[订单管理]
        C4[内容管理]
        C5[站点管理]
        C6[系统管理]
    end
    subgraph 内容区
        D1[商品列表/编辑]
        D2[订单列表/详情]
        D3[文档/推荐位]
        D4[站点设置]
        D5[管理员/部门/角色]
    end
    A -->|登录成功| B
    B --> C1 & C2 & C3 & C4 & C5 & C6
    C2 --> D1
    C3 --> D2
    C4 --> D3
    C5 --> D4
    C6 --> D5
```

仪表板：单页展示核心运营数据与趋势图。数据需求：总用户数、商品数、订单数、销售额等统计（可来自后端统计接口或聚合查询）；用户/订单增长趋势（按日或周）。页面设计：顶部卡片展示关键指标；下方使用 ECharts 折线图或柱状图展示趋势；数据加载态与空态需有占位或提示。

商品管理：包含商品列表、商品新增/编辑、分类管理、商品属性与属性值、标签管理。**商品列表页**：表格展示商品主信息（主图、标题、分类、价格、库存、状态、操作），支持分页（page、pageSize）、筛选（分类、状态、关键词）、排序；操作列含编辑、删除（软删除）。**商品新增/编辑页**：表单含基本信息（标题、分类、主图、轮播图、价格、库存、状态）、SKU 与属性、详情内容（富文本 WangEditor）、标签多选；主图与详情图通过上传组件调用 OSS 接口，上传成功后写入表单字段；提交时组装为后端约定的 DTO（如分类 ID、SKU 列表、属性值 ID、标签 ID、详情 HTML）。**分类管理**：树形表格展示分类树（parent_id 结构），支持展开/折叠、新增节点、编辑、删除；校验父子关系与 code 唯一。**属性与属性值**：列表页展示属性及其值，支持新增/编辑/删除；与商品关联在商品编辑页通过多选或标签形式维护。**标签管理**：列表与表单 CRUD，标签与商品的关联在商品编辑页维护。

订单管理：包含订单列表、订单详情、发货、退款列表与退款操作。**订单列表页**：表格展示订单号、查询码、金额、状态、时间等，支持分页与状态筛选；操作含查看详情、发货、退款。**订单详情页**：展示订单主信息、订单项（商品、SKU、数量、单价）、收货地址、支付信息、状态流转；发货操作为弹窗填写物流公司与单号，提交后调用发货接口。**退款**：列表页展示退款单与关联订单；退款操作由订单详情或列表发起，调用后端退款接口（后端再调 PayPal）。

内容管理（CMS）：包含文档管理、推荐位与推荐项。**文档列表页**：表格展示文档 code、标题、分类、状态、时间，支持分页与筛选；新增/编辑使用 Form + WangEditor，提交时传 content 为 HTML。**推荐位**：列表维护推荐位（如首页轮播、猜你喜欢）；每个推荐位下维护推荐项（关联商品或文档），列表与表单设计需支持类型、排序、跳转链接等。

站点管理：表单页维护站点信息（名称、Logo、联系方式、备案号等）与市场设置（运费、营销开关、前端 baseURL 等）；数据来源为后端 core_config 或 marketSetting 接口，提交时调用更新接口。

系统管理：包含管理员、部门、角色、权限、前台用户管理。**管理员**：列表（与部门、角色关联展示）、新增/编辑（用户名、密码、部门、角色）；**部门**：树形列表与 CRUD；**角色**：列表与 CRUD，角色与权限的绑定通过多选或树形选择；**权限**：列表与 CRUD；**用户管理**：前台用户列表与删除。辅助功能如字典属性、图标查看可用于表单下拉与图标选择。

资源（OSS）：上传组件封装对后端 OSS 接口的调用（单文件/多文件上传），返回文件 URL 或 path 供表单使用；删除与文件信息按需在业务页或统一文件管理中调用。

5.6.3 主要组件与交互设计

通用组件：表格组件封装分页、加载态、空态与操作列；表单组件封装校验规则与提交逻辑；上传组件统一调用 OSS 接口并支持裁剪、大小限制；富文本组件包装 WangEditor，支持图片上传与内容回显。

登录页：表单（用户名、密码），提交调用 `POST /api/manage/core/auth/login`；成功后将返回的 token 存入 Cookie（如 YEX_AUTH）或 localStorage，并跳转至后台首页；失败提示错误信息。

布局：后台采用左侧菜单 + 顶部栏 + 内容区布局。左侧菜单按模块分组（仪表板、商品、订单、内容、站点、系统）；顶部栏含当前用户、退出；内容区为路由出口，子路由对应上述各页面。路由守

卫：除登录页外，访问任意页面前校验本地是否存在 token，不存在则跳转登录。

5.6.4 API 对接设计

认证：除登录接口外，所有请求需携带管理员 Token。设计约定：在 axios 请求拦截器中从 Cookie（YEX_AUTH）或 localStorage 读取 token，设置 Header 的 `Authorization: Bearer <token>` 或 Query 的 `token`；后端管理员认证中间件从上述位置解析 token 并校验 Redis Session。

接口路径与后端对应：core/auth（当前管理员、枚举字典）、core/oss（上传、删除、文件信息）、core/admin、core/dept、core/role、core/permission、shop/product、shop/category、shop/prodAttributes、shop/prodAttributesValue、shop/tag、shop/marketSetting、shop/document、shop/recommend、shop/recommendIndex、shop/order、shop/refund、payment/paypal（退款）、mp/user。请求方法与参数与后端 REST 约定一致（GET 列表/详情、POST 创建、PUT/PATCH 更新、DELETE 删除）。

响应处理：统一根据 `data.code === 0` 判断成功，成功时取 `data.result` 作为业务数据；失败时提示 `data.message`，并根据 code（如 401）决定是否跳转登录页并清除本地 token。

5.7 前台商城详细设计（zhuyi-store）

前台商城面向终端消费者与游客，负责商品浏览、购物车、结算、支付、订单查询、用户与内容展示。本节从技术选型、页面结构、核心流程设计、状态管理与 API 对接四个方面进行详细设计。

5.7.1 技术选型与目录结构设计

技术选型：采用 **Nuxt 4 + Vue 3 + TypeScript**，**Nuxt UI 4 + Tailwind CSS 4** 作为 UI 与样式，**pnpm** 作为包管理。后端对接通过 **runtimeConfig.public.apiUrl** 配置（默认 `http://localhost:8080/api/client`），与后端 `/api/client` 对应。支持 **SSR 与预渲染**（如首页设置 `prerender: true`），有利于 SEO 与首屏性能。

目录结构设计：**app/api** 存放接口封装与类型定义（index.ts 按 shop、blogs 等模块组织，type.ts 定义请求/响应与实体类型）；**app/utils** 存放 request 封装、auth（设备指纹、Cookie

accessToken）、图片工具；**app/composables** 存放组合式逻辑（如 useCartShared、useAddressData）；**app/hook** 存放业务 Hook（如 useCartHook 加购）；**app/layouts** 存放 default 布局；**app/pages** 存放页面（文件系统路由：index、collections/[code]、product/[id]、cart、checkout、order-detail/[id]、blogs/[code]、login、register、account/addresses）；**app/components** 存放 ProductCard、ProductGrllery、ProductSkuBox、BaseNav 等；**data/address.json** 存放国家/省数据，供结算与地址簿的国家/省联动使用。请求封装（app/utils/request.ts）：baseURL 为 runtimeConfig.public.apiUrl；请求头需携带 `Authorization`（从 Cookie accessToken 读取）、`X-Device-Fingerprint`（从 auth 获取设备指纹）；响应解析约定为 `data.code === 0` 时返回 `data.result`，否则抛出错误或 reject。

5.7.2 页面结构与路由设计

页面与路由：**首页** `/`：商品列表与分页，可选推荐位区块；**分类集合页** `/collections/[code]`：按分类 code 展示该分类下商品，分页，SEO 使用分类的 seo_title、seo_description；**商品详情页** `/product/[id]`：主图/画廊、价格、SKU 选择、属性、富文本详情、规格参数 Tab、加购按钮、“猜你喜欢”；**购物车** `/cart`：购物车列表、改数量、删除、小计/合计、跳转结算；**结算页** `/checkout`：收货信息表单、支付方式（PayPal）、创建订单并跳转 PayPal；**订单详情** `/order-detail/[id]`：按订单号或查询码展示订单信息、商品、金额、收货地址、状态；**博客/文档列表**：在 layout 的 Footer 通过文档列表接口拉取多列链接，链到 `/blogs/[code]`；**文档详情** `/blogs/[code]`：按 code 展示标题、时间、作者、阅读数、正文（富文本）、来源等；**登录** `/login`、**注册** `/register`：表单提交后写 Cookie accessToken，支持 redirect 回跳；**地址簿** `/account/addresses`：地址列表、新增/编辑/删除、设默认，国家/省选项来自 address.json。前台商城页面与 API 的对应关系如表 5-6 所示。

表 5-6 前台商城页面与 API 对照表

页面	路由	主要 API 调用
首页	/	shop.product.list, blogs.recommend
分类集合	/collections/[code]	shop.category, shop.product.list
商品详情	/product/[id]	shop.product 详情, 推荐
购物车	/cart	

		shop.userCart 列表/改数量/删除
结算	/checkout	shop.order.create, payment PayPal
订单详情	/order-detail/[id]	shop.order 按查询码/订单号
文档详情	/blogs/[code]	blogs.document.info
登录/注册	/login, /register	shop.userAuth 登录/注册
地址簿	/account/addresses	shop.userAddress CRUD

图 5-3 前台商城页面路由与布局结构图

代码块

```
flowchart TB
    subgraph 布局 default
        H[Header: Logo/Nav/购物车/用户]
        M[面包屑]
        F[Footer: 文档链接]
    end
    subgraph 页面
        P1[/ 首页]
        P2[/collections 分类]
        P3[/product 商品详情]
        P4[/cart 购物车]
        P5[/checkout 结算]
        P6[/order-detail 订单]
        P7[/blogs 文档]
        P8[/login /register]
        P9[/account/addresses]
    end
    H --> P1 & P2 & P3 & P4
    M --> P2 & P3
    P1 --> P3
    P3 --> P4
    P4 --> P5
    P5 --> P6
    F --> P7
```

布局（layouts/default.vue）：Header 含 Logo、BaseNav（首页 + 分类下拉）、搜索（关键词带到首页 query）、购物车图标（显示 cartNum）、用户下拉（未登录：登录/注册；已登录：购物车、退出）。面包屑：非首页时根据当前路由（分类页/商品页）调用 market.breadcrumb 接口动态生成。Footer：从文档列表生成多列链接，无数据时显示默认区块（关于我们、客户服务等）。鉴权：通过 Cookie accessToken 判断是否登录，控制用户菜单与跳转。

5.7.3 核心流程设计

商品浏览与加购流程：首页/分类页加载商品列表（分页）；点击商品进入详情页，加载商品详情（含 SKU、属性、详情内容）；用户选择 SKU 与数量后点击“加入购物车”，调用 useCartHook.addCart(product_id, sku_id, quantity)；若未登录，可设计为允许加购（依赖设备指纹）并提示“登录后可在多端同步”，或提示先登录再加购；加购成功后刷新购物车数量（useCartShared.refreshAllCartData、useState('cartNum')）并可选提示成功。

购物车与结算流程：购物车页加载购物车列表（根据 userID 或设备指纹）；用户可修改数量、删除项；点击“去结算”跳转结算页。结算页：展示收货信息表单（姓名、邮箱、电话、国家/省/城市/邮编、街道），国家/省联动使用 address.json；支付方式为 PayPal；校验表单通过后调用 order.create 创建订单，再调用 payment 获取 PayPal 支付 URL，前端跳转至 PayPal 完成支付；支付成功后 PayPal 回调至后端，后端 capture 后重定向到前端订单详情页（或成功页），前端按订单号或查询码展示订单详情。

订单查询流程：已登录用户可通过“我的订单”进入订单列表；游客通过“订单查询”输入查询码，结合设备指纹调用订单详情接口，后端校验归属后返回订单信息；订单详情页展示订单项、金额、收货地址、状态（待支付/已支付/已发货/已退款等）。

用户与地址流程：注册页提交邮箱、用户名、密码，成功后跳转登录；登录页提交邮箱、密码，成功后写 Cookie accessToken 并支持 redirect；地址簿页调用 address 列表、新增、修改、删除接口，国家/省选项与 address.json 一致，设默认等操作调用对应接口。

5.7.4 状态管理与 API 对接设计

购物车状态：使用 **useState('cartNum')** 存储购物车数量，供 Header 购物车图标展示；**useCartShared** 提供 useCartList（购物车列表）、refreshAllCartData（刷新列表与数量）；在加购、删除、登录后调用 refreshAllCartData，保证导航栏数量与购物车页列表一致。设计要点：加购/

删除接口成功后必须调用 refreshAllCartData；若支持游客加购，设备指纹需在首次访问时生成并写入 Cookie（如 device_id），请求时通过 X-Device-Fingerprint 传给后端。

设备指纹与游客：utils/auth 提供 getDeviceFingerprint（Cookie device_id 或浏览器指纹），在 request 拦截器中统一附加到 Header；后端设备指纹中间件写入上下文，购物车与订单按 userID 或 deviceFingerprint 关联。登录态用 Cookie accessToken，可选认证中间件解析后写入 userID，需登录的接口由后端校验。

API 模块划分：shop.category（分类树、分类详情、按 code/父级）、shop.product（商品列表、商品详情）、shop.user（注册、登录、当前用户、重置密码等）、shop.cart（加购/改数量/删、购物车列表）、shop.address（地址 CRUD）、shop.order（创建订单、订单列表、按查询码查单）、shop.market（面包屑、站点信息、运费）、shop.tag（标签详情、标签下商品）、blogs.document（文档列表、文档详情）、blogs.recommend（推荐位内容）。类型定义集中在 app/api/type.ts（商品、SKU、订单、地址、文档等），与后端返回结构一致，便于类型安全与联调。

5.8 配置与启动流程

配置加载：**config/config.go** 使用 Viper 读取 **config.yaml**，结构体包含 server（端口、运行模式）、database（MySQL）、redis、email、payment（PayPal client_id、secret、api_base）、frontend（前端 URL）等。启动时先加载配置，再初始化 MySQL 连接（GORM）、Redis 连接（go-redis）。

启动流程（cmd/main.go）：加载 config → 初始化 DB、Redis → 创建 RepositoryFactory、ServiceFactory → 注册路由（传入 Factory 或已创建的 Handler）→ 启动 HTTP 服务（如 `r.Run(":8080")`）。若需自动迁移表结构，可在 main 中调用 `db.AutoMigrate(...)` 按模型迁移。启动流程如图 5-4 所示。

图 5-4 后端启动流程图

代码块

```
flowchart TD
    A[main.go 启动] --> B[Viper 加载 config.yaml]
    B --> C[初始化 MySQL 连接]
    C --> D[初始化 Redis 连接]
    D --> E[创建 RepositoryFactory]
```

```
    E --> F[创建 ServiceFactory]
    F --> G[注册路由]
    G --> H[可选: AutoMigrate]
    H --> I[r.Run 启动 HTTP 服务]
```

5.9 详细设计小结

维度	设计要点
分层架构	Handler（解析请求、调Service、写响应）→ Service（业务规则、事务、外部调用）→ Repository（CRUD、查询）；Factory 统一提供 Repository 与 Service。
路由与中间件	/api 下公开登录、/api/manage（管理员认证）、/api/client（设备指纹+可选 JWT）；CORS、Session、JWT、设备指纹中间件。
核心模块	core（管理员、部门、角色、权限、配置）、商品与店铺（商品、分类、属性、标签、SKU、市场）、订单与支付（订单、退款、PayPal）、用户与端侧（用户、地址、购物车）、CMS（文档、推荐）、OSS。
接口与数据	统一响应（code、message、result）；分页（page、pageSize，list+total）；关键模型与表与第四章对应，GORM 映射与关联。
管理后台	Vue3+Vite+Ant Design Vue+Pinia；模块划分（仪表

	板、商品、订单、CMS、站点、系统、OSS）；主要页面与组件（表格、表单、上传、富文本、树形、路由守卫）；API 对接（token 携带、路径与 /api/manage 对应、响应处理）。
前台商城	Nuxt 4+Nuxt UI+Tailwind；页面结构（首页、分类、商品详情、购物车、结算、订单、博客、登录注册、地址）；核心流程（浏览加购、购物车结算、订单查询、用户与地址）；状态管理（useState cartNum、useCartShared、设备指纹、accessToken）；API 对接（apiUrl、shop/blogs 模块、type.ts）。
配置与启动	config.yaml + Viper；main 中初始化 DB/Redis、Factory、路由并启动服务。

本章对后端分层、路由与中间件、核心业务模块、关键接口与数据模型进行了详细设计，并补充了管理后台与前台商城的详细设计（技术选型、模块/页面结构、主要组件与流程、API 对接），为第六章系统实现提供完整设计依据。

第六章 系统实现

系统实现是在详细设计的基础上，将设计转化为可运行的代码与可部署的应用。本章从**后端服务（server）**、**管理后台（shop_admin）**与**前台商城（zhuyi-store）**三个子系统分别阐述实现要点，包括技术选型、目录结构、关键实现与 API 对接，以体现系统整体从设计到落地的过程。

6.1 系统实现概述

（1）实现环境。 后端：Go 1.24+、MySQL、Redis，开发与运行环境为 Windows / macOS / Linux；前端：Node.js（推荐 LTS）、pnpm 或 npm。开发时后端默认端口 8080，管理后台默认端口 3001，前台商城默认端口 3000；通过环境变量或配置文件指定 API 基地址，实现前后端联调。

（2）技术栈汇总。 后端：Go + Gin + GORM + MySQL + Redis + Viper + PayPal SDK + gomail；管理后台：Vue 3 + TypeScript + Vite + Ant Design Vue 4.x + Pinia + Axios + ECharts + WangEditor；前台商城：Nuxt 4 + Vue 3 + Nuxt UI 4 + Tailwind CSS + TypeScript。各子系统的技术选型与第四章总体设计、第五章详细设计一致。

（3）部署方式。 后端编译为单一可执行文件，依赖 config.yaml、MySQL、Redis 即可运行；管理后台与前台商城分别执行 `npm run build` 或 `pnpm build` 生成静态资源或 Node 服务，通过 Nginx 或 Node 托管，并将 API 请求代理至后端。毕业设计演示阶段可在本机同时启动三个子项目，通过本地 API 基地址完成联调与演示。

6.2 后端服务实现

6.2.1 入口与配置实现

入口（cmd/main.go）：程序启动时依次执行：加载配置（config.Load 或 Viper 读取 config.yaml）→ 初始化 MySQL 连接（GORM Open，并设置连接池与日志）→ 初始化 Redis 连接（go-redis NewClient）→ 创建 RepositoryFactory（传入 *gorm.DB）、ServiceFactory（传入 RepositoryFactory 或各 Repository）→ 调用 router.Register 注册路由（传入 Gin 引擎与 Factory）→ 启动 HTTP 服务（r.Run(":8080")）。可选：在 main 中调用 db.AutoMigrate 按模型自动迁移表结构，便于开发与演示。

配置（config/config.go、config.yaml）：使用 Viper 绑定 YAML，结构体包含 Server（Port、Mode）、Database（Host、Port、Username、Password、DBName、Charset）、Redis（Addr、Password、DB）、Email（SMTP 配置）、Payment（PayPal ClientID、Secret、APIBase，支持 sandbox）、Frontend（BaseURL 等）。配置在 main 中加载后传入需要使用的包（如 middleware、service），实现不同环境切换（开发/生产）时仅改配置文件。

6.2.2 路由与中间件实现

路由注册（router/router.go）：创建 Gin 引擎，使用 gin.Default() 或 gin.New()；挂载全局 CORS 中间件（设置 AllowOrigin、AllowMethods、AllowHeaders、AllowCredentials）。创建 `/api` 路由组，在其下挂载：① 公开路由：POST `/api/manage/core/auth/login`，绑定 AdminAuthHandler.Login；② 管理端路由组：前缀 `/api/manage`，挂载 AdminAuthMiddleware，其下按业务挂子组（如 core/auth、core/admin、core/dept、core/role、core/permission、core/oss、shop/product、shop/category、shop/prodAttributes、shop/prodAttributesValue、shop/tag、shop/marketSetting、shop/document、shop/recommend、shop/recommendIndex、shop/order、shop/refund、payment/paypal、mp/user），每个子组绑定对应 Handler 的 GET/POST/PUT/DELETE；③ 客户端路由组：前缀 `/api/client`，先挂载 DeviceFingerprintMiddleware，再挂载 OptionalClientAuthMiddleware，其下挂子组（shop/document、shop/product、shop/category、shop/market、shop/tag、shop/recommendIndex、shop/userCart、shop/userAuth、shop/order、shop/userAddress、payment），绑定对应 Handler。静态资源：`/api/oss` 使用 gin.Static 或 StaticFS 映射到本地目录（如 ./oss）。

中间件实现（middleware/）：CORS 中间件：在 c.Writer.Header() 中设置 Access-Control-Allow-Origin 等，若 c.Request.Method == "OPTIONS" 则 c.AbortWithStatus(200) 并 return，否则 c.Next()。管理员认证中间件：从 c.GetHeader("Authorization")、c.Query("token")、c.Cookie("YEX_AUTH") 解析 token，调用 utils.GetSession(token) 查 Redis，若 Session 存在则 c.Set("user", ...)、c.Set("userID", ...)、c.Set("token", token)，c.Next()；否则 c.AbortWithStatusJSON(401, 统一错误格式)。设备指纹中间件：从 c.GetHeader("X-Device-Fingerprint")、Query、Cookie 读取，若无则生成 UUID，c.Set("deviceFingerprint", fingerprint)，c.Next()。可选认证中间件：从 Authorization Bearer 解析 JWT，若有效则解析 claims 并 c.Set("userID", ...)、c.Set("email", ...)，c.Next()；无效或未携带则直接 c.Next()，不 Abort。

6.2.3 分层与关键实现要点

Handler 层（handlers/admin、handlers/client）：每个接口对应一个 Handler 函数，接收 *gin.Context；从 c.Get("userID")、c.Get("deviceFingerprint") 等读取上下文；解析 Query、Body 参数；调用 ServiceFactory 获取对应 Service，调用 service.XXX()；根据返回 error 或 data 调用 utils.Success(c, result) 或 utils.Fail(c, code, message)。统一响应在 utils 中实现：Success 写 {"code":0,"message":"ok","result":data}，Fail 写 {"code":code,"message":message}。

Service 层（service/）：各业务 Service 在构造函数中注入所需 Repository；业务方法接收 DTO 或 ID，校验业务规则后调用 Repository 读写数据库；涉及多表写操作时使用 db.Transaction，在回调中调用多个 Repository 方法；与 PayPal、邮件等外部调用在 Service 内完成，不放在 Handler 中。

Repository 层（repository/）：各 Repository 接收 *gorm.DB（或 tx），实现 GetByID、List、Create、Update、Delete 等方法；List 支持分页（Offset、Limit）与筛选（Where）；复杂查询使用 Preload 加载关联，避免 N+1。Factory 在创建时传入 db，返回实现接口的 Repository 实例。

数据模型（models/）：按业务域分目录（common、core、cms、mp、shop、sp、mypaypal），每个模型对应一张表，使用 GORM 的 TableName、主键、CreatedAt/UpdatedAt；关联使用 ForeignKey、References 或手动维护外键字段。密码等敏感字段在 Service 层加密后写入，查询结果不返回明文。

6.3 管理后台实现（shop_admin）

6.3.1 技术选型与目录结构

技术选型：Vue 3 + TypeScript + Vite 作为基础框架，Ant Design Vue 4.x 作为 UI 组件库，Pinia 作为状态管理，Axios 作为 HTTP 客户端，ECharts 用于仪表板图表，WangEditor 用于富文本编辑（如商品详情、CMS 文档）。构建工具为 Vite，默认端口 3001，API 基地址通过环境变量 VITE_API_BASE_URL 配置（如 http://localhost:8080/api），实际请求为 VITE_API_BASE_URL + /manage/...。

目录结构：src/api 存放接口封装（按模块分文件，如 auth、product、order、cms、site、system、oss）；src/components 存放通用组件（如表格、表单、上传）；src/views 存放页面视图（如 Dashboard、Product、Order、Cms、Site、System）；src/router 配置路由（登录页、布局与子路由）；src/store 使用 Pinia 存储用户信息、Token 等；vite.config.ts 配置代理与环境变量。请求封装：在 src/utils 或 src/api 中封装 axios 实例，baseURL 为 VITE_API_BASE_URL，请求拦截器中从 Cookie 或 localStorage 读取 token 并写入 Header（Authorization: Bearer <token>），响应拦截器中根据 code 判断成功与否，非 0 时统一提示并可选跳转登录。

6.3.2 主要页面与组件实现

登录页：表单（用户名、密码），提交后调用 POST /api/manage/core/auth/login，成功后将返回的 token 存入 Cookie 或 localStorage，并跳转至后台首页；失败则提示错误信息。

仪表板（Dashboard）：调用统计接口（如用户数、商品数、订单数、销售额）与趋势接口（用户/订单增长），使用 ECharts 渲染折线图或柱状图；卡片展示核心指标。

商品管理：列表页使用 Ant Design Vue 的 Table，支持分页、筛选（分类、状态）、搜索；新增/编辑页使用 Form 与动态表单项，商品主图与详情图使用上传组件（调用 OSS 接口），详情内容使用 WangEditor；提交时调用商品创建/更新接口，关联分类、SKU、属性值、标签等在提交前组装为后端约定格式。分类管理：树形表格展示分类树，支持新增/编辑/删除节点。

订单管理：列表页 Table 展示订单号、状态、金额、时间等，支持状态筛选与分页；详情页展示订单项、收货地址、支付信息；发货操作用 Modal 填写物流单号，调用发货接口；退款操作调用退款接口（后端再调 PayPal 退款）。

内容管理（CMS）：文档列表 Table，新增/编辑使用 Form + WangEditor；推荐位与推荐项分别维护，推荐项可关联商品或文档。

站点管理：表单维护站点信息（名称、Logo、联系方式等）与市场设置（运费、营销开关等），提交后调用市场设置接口。

系统管理：管理员、部门、角色、权限、用户管理分别对应列表与表单页，调用 core/admin、core/dept、core/role、core/permission、mp/user 等接口；字典属性、图标查看为辅助功能。

6.3.3 API 对接

所有管理端请求均需携带管理员 Token：在 axios 请求拦截器中从 Cookie（如 YEX_AUTH）或 localStorage 读取 token，设置 Header 的 Authorization: Bearer <token> 或 Query 的 token。接口路径与后端一致：如 core/auth（当前管理员、枚举字典）、core/oss（上传、删除）、shop/product、shop/category、shop/order、shop/document、shop/recommend、payment/paypal（退款）、mp/user 等。响应处理：根据 data.code === 0 判断成功，取 data.result 作为业务数据；否则提示 data.message 并根据 code 决定是否跳转登录（如 401）。

6.4 前台商城实现（zhuyi-store）

6.4.1 技术选型与目录结构

技术选型：Nuxt 4 + Vue 3 + TypeScript，Nuxt UI 4 + Tailwind CSS 4 作为 UI 与样式，pnpm 作为包管理。后端对接通过 runtimeConfig.public.apiUrl 配置（默认 http://localhost:8080/api/client），请求时携带设备指纹与可选 JWT。支持 SSR 与预渲染（如首页设置 prerender: true），有利于 SEO 与首屏性能。

目录结构：app/api 存放接口封装（index.ts 中按 shop、blogs 等模块组织，如 shop.product、shop.category、shop.cart、shop.order、shop.userAuth、shop.userAddress、shop.market、shop.tag、blogs.document、blogs.recommend）与类型定义（type.ts）；app/utils 存放 request 封装、auth（设备指纹、Cookie accessToken）、图片工具；app/composables 存放 useCartShared、useAddressData 等组合式逻辑；app/hook 存放 useCartHook（加购）；app/layouts 存放 default 布局；app/pages 存放页面（index、collections/[code]、product/[id]、cart、checkout、order-detail/[id]、blogs/[code]、login、register、account/addresses）；app/components 存放 ProductCard、ProductGrllery、ProductSkuBox、BaseNav 等；data/address.json 存放国家/省数据供结算与地址簿使用。

请求封装（app/utils/request.ts）：基于 fetch 或 $fetch，baseURL 为 runtimeConfig.public.apiUrl；请求头设置 Authorization（从 Cookie accessToken 读取）、X-Device-Fingerprint（从 auth 获取设备指纹）；响应解析：若 data.code === 0 则返回 data.result，否则抛出错误或返回 Promise.reject。

6.4.2 主要页面与流程实现

首页（pages/index.vue）：调用 api.shop.product.list 获取商品列表与分页；可选调用 blogs.recommend 获取推荐位内容；使用 ProductCard 展示商品列表；支持预渲染。

分类集合页（pages/collections/[code].vue）：根据路由参数 code 调用 shop.category 按 code 查询分类信息，再调用 shop.product.list 传入分类条件，分页展示商品；SEO 使用分类的 seo_title、seo_description 等。

商品详情页（pages/product/[id].vue）：调用 shop.product 详情接口，展示主图/画廊（ProductGrllery）、价格、SKU 选择（ProductSkuBox）、属性、富文本详情、规格参数 Tab；加购按钮调用 useCartHook.addCart(product_id, sku_id, quantity)，未登录时提示并跳转登录；“猜你喜欢”根据商品标签调用推荐接口展示关联商品。

购物车（pages/cart.vue）：调用 shop.userCart 列表接口，展示购物车项、改数量、删除；使用 useCartShared 的 useCartList、refreshAllCartData 与 useState('cartNum') 保持导航栏数量与列表一致；小计/合计计算后提供跳转结算入口。

结算页（pages/checkout.vue）：收货信息表单（姓名、邮箱、电话、国家/省/城市/邮编、街道），国家/省联动使用 address.json；支付方式为 PayPal；校验通过后调用 shop.order.create 创建订单，再调用 payment 获取 PayPal 支付 URL，跳转至 PayPal 完成支付。

订单详情（pages/order-detail/[id].vue）：根据订单号或查询码调用 shop.order 详情接口，展示订单信息、商品、金额、收货地址、状态；游客通过查询码与设备指纹校验归属后查看。

登录/注册（pages/login.vue、register.vue）：登录表单（邮箱、密码），成功后写 Cookie accessToken，支持 redirect 跳回；注册表单（邮箱、用户名、密码），成功后跳转登录。

地址簿（pages/account/addresses.vue）：调用 shop.userAddress 列表、新增、修改、删除接口；使用 useAddressData 与 address.json 提供国家/省选项；设默认等操作调用对应接口。

博客/文档（layouts 中 Footer 文档列表、pages/blogs/[code].vue）：Footer 通过 api.blogs.document.list 拉取文档，多列展示并链到 /blogs/[code]；文档详情页按 code 调用 api.blogs.document.info，展示标题、时间、作者、阅读数、正文（富文本）、来源等。

6.4.3 布局与全局状态

默认布局（layouts/default.vue）：Header 含 Logo、BaseNav（首页 + 分类下拉）、搜索（关键词带到首页 query）、购物车图标（显示 cartNum）、用户下拉（未登录：登录/注册；已登录：购物车、退出）。面包屑：非首页时根据当前路由（分类页/商品页）调用 api.shop.market.breadcrumb 动态生成。Footer：从文档列表生成多列链接，无数据时显示默认区块（关于我们、客户服务等）。鉴权：通过 Cookie accessToken 判断是否登录，控制用户菜单与跳转。

购物车状态：使用 useState('cartNum') 存储购物车数量，useCartShared 提供 useCartList、refreshAllCartData，在加购、删除、登录后调用 refreshAllCartData 刷新，保证导航栏与购物车页数据一致。

设备指纹与游客：utils/auth 提供设备指纹（Cookie device_id 或浏览器指纹），请求时通过 X-Device-Fingerprint 传给后端，用于游客购物车与订单关联；登录态用 Cookie accessToken，OptionalClientAuthMiddleware 解析后写入上下文，需登录的接口由后端校验 userID。

6.5 系统实现小结

子系统	实现要点
后端 server	入口 main.go 加载配置、初始化 DB/Redis、Factory、注册路由并启动服务；路由按公开、/api/manage、/api/client 分组并挂载 CORS、管理员认证、设备指纹、可选 JWT 中间件；Handler→Service→Repository 分层，统一响应与错误处理；核心模块与第五章详细设计一致。
管理后台 shop_admin	Vue3+Vite+Ant Design Vue+Pinia；src/api、views、router、store；登录、仪表板、商品/订单/CMS/站点/系统管理、OSS 上传；axios 拦截器携带 token，接口路径与后端 /api/manage 对应。
前台商城 zhuyi-store	Nuxt 4+Nuxt UI+Tailwind；app/api、pages、layouts、composables、utils；首页、分类、商品详情、购物车、结算、订单详情、登录注册、地址、博客；request 携带设备指纹与 accessToken，接口与 /api/client 对应；useCartShared、useState('cartNum')、address.json 支持购物车与结算流程。

本章从后端、管理后台与前台商城三个子系统阐述了系统实现要点，包括入口与配置、路由与中间件、分层实现、主要页面与组件、API 对接与全局状态管理，体现了从设计到可运行系统的完整实现过程。以下第七章对系统进行测试并总结全文。

第七章 系统测试与总结

系统测试是验证系统是否满足需求、能否稳定运行的重要环节；总结则对课题工作、不足与后续方向进行归纳。本章先给出测试环境与测试方案，再对主要功能与核心流程进行测试说明并归纳测试结果，最后对本课题的工作进行总结，指出不足与改进方向。

7.1 测试环境与测试方案

7.1.1 测试环境

硬件与系统：开发与测试在普通 PC 或笔记本上进行，操作系统为 Windows 10/11 或 macOS；毕业设计演示规模下未对服务器配置做特殊要求。

软件环境：后端：Go 1.24+、MySQL 8.x、Redis 6.x，后端服务默认端口 8080；管理后台：Node.js 18+、Vue 3、Vite，默认端口 3001；前台商城：Node.js 18+、Nuxt 4，默认端口 3000。API 基地址：管理后台配置为 `http://localhost:8080/api`（或实际后端地址），前台商城配置为 `http://localhost:8080/api/client`。PayPal 使用沙箱环境（sandbox）进行支付与退款测试，无需真实扣款。

数据准备：测试前初始化数据库（执行迁移或 SQL 脚本），插入必要的管理员账号、分类、商品（含 SKU）、站点配置等基础数据；Redis 用于会话，无需预置数据。

7.1.2 测试方案

测试范围：覆盖管理端、客户端与后端核心功能，重点验证：① 管理员登录与后台各模块的增删改查；② 前台商品浏览、购物车、结算、PayPal 支付与订单查询；③ 用户注册登录、地址管理；④ CMS 文档与推荐位在前台的展示；⑤ 接口认证（管理员 Session、用户 JWT、设备指纹）与统一响应格式。

测试方法：采用**黑盒测试**为主，按业务场景从前端操作到后端接口进行端到端验证；关键接口可辅以**接口测试**（如 Postman 或 curl）验证参数与响应。不单独编写自动化测试脚本时，以**手工测试**与**演示用例**为主，记录通过/失败与简要现象。

图 7-1 系统测试流程图 测试流程如图 7-1 所示：搭建测试环境后，依次对管理端、客户端与后端接口进行功能测试，重点验证核心业务流程（如购物流程、支付流程）；汇总测试结果并记录问题，通过则进入总结阶段。

代码块

```
flowchart TD
    A[搭建测试环境] --> B[管理端功能测试]
    B --> C[客户端功能测试]
    C --> D[后端接口与认证测试]
    D --> E[核心流程端到端测试]
    E --> F{测试通过?}
    F -->|否| G[记录问题并修复]
    G --> B
    F -->|是| H[汇总测试结果]
    H --> I[编写测试报告]
```

测试类型：**功能测试**（各模块功能是否符合需求）、**流程测试**（核心业务流程是否贯通，如加购→结算→支付→订单详情）、**兼容性测试**（前后端联调、不同浏览器访问前台商城）。性能与压力测试在毕业设计阶段不作为必选，若有需要可对关键接口做简单并发或响应时间观察。

7.2 功能测试

7.2.1 管理端功能测试

登录与认证：在管理后台登录页输入正确管理员账号与密码，提交后应跳转至后台首页，且后续请求携带 Token；使用错误密码或未登录访问需认证的页面应被拒绝或跳转登录。**仪表板**：进入仪表板后应展示用户数、商品数、订单数、销售额等统计及趋势图，数据与数据库或后端统计接口一致。**商品管理**：商品列表分页、筛选、搜索正常；新增商品时填写基本信息、分类、SKU、详情、标签并上传图片，提交后列表中可见新商品；编辑、软删除功能正常；分类树展示与增删改查正常；属性与属性值、标签管理正常。**订单管理**：订单列表分页与状态筛选正常；订单详情展示订单项、收货地址、状态；发货操作填写物流信息后订单状态更新；退款操作可发起并调用后端退款接口（PayPal 沙箱）。**内容管理（CMS）**：文档列表与新增/编辑/删除正常，富文本与图片上传正常；推荐位与推荐项增删改查正常。**站点管理**：站点信息与市场设置保存后，前台能读取到更新后的站点信息、运费等。**系统管理**：管理员、部门、角色、权限、前台用户列表与 CRUD 正常。**OSS**：文件上传、删除、访问正常，商品与文档中引用的图片可正常显示。

测试结果：在上述测试用例下，管理端各模块功能符合需求，与后端 `/api/manage` 接口对接正常；认证失败时能正确跳转登录或提示。

7.2.2 客户端功能测试

商品与分类：首页商品列表与分页正常；分类集合页按分类 code 展示对应商品；商品详情页展示主图、价格、SKU 选择、属性、富文本详情，“猜你喜欢”能展示关联商品。**购物车**：加购后购物车数量与列表更新；修改数量、删除项正常；未登录用户加购依赖设备指纹，同一设备下购物车一致；登录后购物车可与用户关联（视业务实现）。**结算与支付**：结算页收货信息表单与国家/省联动正常；提交后能创建订单并跳转 PayPal 沙箱；在沙箱中完成支付后，后端能捕获订单并更新状态，前端能跳转至订单详情或成功页。**订单查询**：已登录用户可查看订单列表与详情；游客通过查询码与设备指纹可查看自己的订单。**用户与地址**：注册、登录、退出正常，登录态写入 Cookie 后菜单与权限正确；地址簿增删改查、设默认正常，国家/省选项与 address.json 一致。**内容（博客）**：Footer 文档列表与链接正常；文档详情页按 code 展示标题、正文等。**布局与状态**：Header 购物车数量、用户下拉、面包屑、Footer 与登录态一致；设备指纹在请求中正确携带。

测试结果：客户端主要页面与流程符合需求，与后端 `/api/client` 接口对接正常；游客与登录用户的购物车、订单查询行为符合设计。

7.2.3 后端接口与认证测试

公开接口：`POST /api/manage/core/auth/login` 使用正确账号密码返回 token，错误密码返回失败信息。**管理端接口**：未携带 Token 或 Token 无效时访问 `/api/manage/*` 返回 401；携带有

效 Token 后，core/auth、shop/product、shop/order 等接口返回正确数据与统一格式（code、message、result）。**客户端接口**：未携带设备指纹时可由中间件生成或拒绝（按实现）；携带设备指纹与可选 JWT 后，shop/product、shop/userCart、shop/order 等接口按业务返回数据；需登录的接口在未登录时返回相应错误。**统一响应**：成功时 code 为 0、result 为业务数据；失败时 code 非 0、message 为错误说明；分页接口返回 list 与 total。

测试结果：接口分组与认证行为与设计一致，统一响应格式便于前端处理；PayPal 沙箱环境下创建订单、捕获订单与退款流程可走通。

7.3 核心流程测试与测试结果小结

核心流程：① **管理端**：登录 → 仪表板 → 商品新增/编辑 → 订单列表/发货/退款 → CMS 文档/推荐位 → 站点设置；② **客户端**：首页/分类 → 商品详情 → 加购 → 购物车 → 结算（收货信息）→ 创建订单 → PayPal 支付 → 订单详情；③ **游客**：加购（设备指纹）→ 结算 → 订单查询（查询码+设备指纹）。上述流程在测试环境中按步骤执行，各环节数据一致、状态正确、前后端联调通过。

测试结果小结：在给定测试环境与测试方案下，系统主要功能与核心流程**通过**测试，满足毕业设计演示与答辩需求。管理端、客户端与后端接口行为与第三、四、五章需求与设计一致；认证、统一响应与 API 对接符合预期。PayPal 使用沙箱环境，真实生产环境需切换配置并遵守 PayPal 与当地合规要求。未进行大规模性能与安全渗透测试，若系统投入实际使用，建议补充性能测试、安全加固与更完整的异常与边界用例测试。

7.4 工作总结

本课题设计并实现了一套**基于前后端分离的跨境电商独立站系统**，包括后端 API 服务（server）、管理后台（shop_admin）与前台商城（zhuyi-store）三个子系统，完成了从需求分析、总体设计、详细设计到系统实现与测试的完整软件工程流程。

（1）需求与设计。 在引言与可行性分析的基础上，完成了系统需求分析（用户角色、管理端与客户端功能、后端 API 与业务逻辑、非功能需求）；进行了系统总体设计（总体架构、前后端分离与通信、子系统划分、认证与安全、数据库与接口总体设计）；在详细设计阶段对后端分层架构、路由与中间件、核心业务模块、关键接口与数据模型进行了细化，并补充了管理后台与前台商城的详细设计（技术选型、模块与页面结构、主要组件与流程、API 对接），为实现提供了清晰的设计依据。

（2）系统实现。 后端采用 Go + Gin 实现了 Handler → Service → Repository 分层架构，通过 Factory 统一管理仓储与业务服务，完成了核心系统、商品与店铺、订单与支付、用户与端侧、CMS、资源与配置等模块，提供 `/api/manage` 与 `/api/client` 双端接口，集成 Redis Session、JWT、设备指纹、PayPal 与邮件。管理后台基于 Vue 3 + Vite + Ant Design Vue 实现了仪表板、商品管理、订单管理、内容管理、站点管理、系统管理与 OSS 上传，与管理端 API 对接。前台商城基于 Nuxt 4 + Nuxt UI 实现了首页、分类、商品详情、购物车、结算、订单详情、登录注册、地址簿与博客等页面与流程，支持设备指纹下的游客购物车与订单查询，并与客户端 API 及 PayPal 支付对接。

（3）测试与验证。 在给定测试环境下，对管理端、客户端与后端进行了功能测试与核心流程测试，主要功能与业务流程通过验证，系统可部署、可演示，满足毕业设计答辩要求。

通过本课题，将软件工程中的需求分析、总体设计、详细设计、实现与测试等环节贯穿于一个完整的独立站项目中，对前后端分离架构、分层设计、RESTful API、多端认证与支付集成有了更深入的理解与实践。

7.5 不足与改进方向

（1）功能与业务。 当前系统聚焦于独立站核心链路，部分功能可进一步扩展：如优惠券、促销活动、会员等级、评价与晒单、多语言与多币种、物流跟踪与消息通知等；搜索目前多为列表筛选，可引入全文检索（如 Elasticsearch）以提升搜索体验。

（2）性能与可用性。 未系统开展性能测试与压测，在高并发场景下的响应时间与稳定性有待验证；可引入缓存策略（如商品详情、分类树）、数据库索引优化与读写分离、静态资源 CDN 等提升性能与可用性。

（3）安全与合规。 已实现密码加密、Session/JWT、越权防护与 PayPal 服务端校验，但未进行专门的安全审计与渗透测试；生产环境建议加强 HTTPS、防重放、限流、日志与审计，并对用户隐私与支付合规做专门设计。

（4）运维与监控。 当前部署与配置以手工为主，可引入 Docker 与编排（如 Docker Compose、K8s）、CI/CD 与健康检查、日志聚合与监控告警，便于运维与问题排查。

（5）测试与质量。 当前以手工测试与演示为主，可补充单元测试（如后端 Service、Repository）、接口自动化测试与前端 E2E 测试，形成回归能力，提高交付质量。

7.6 展望

本系统作为毕业设计项目，已实现跨境电商独立站的核心功能与完整流程，具备一定的实用性与扩展空间。后续可在上述不足与改进方向的基础上，结合业务需求逐步完善功能、性能、安全与运维，使其更贴近实际生产应用；同时也可将本项目作为学习前后端分离架构、Go 后端开发与 Vue/Nuxt 前端开发的实践案例，持续迭代与总结。

参考文献

[1] 张海藩, 牟永敏. 软件工程导论[M]. 6 版. 北京: 清华大学出版社, 2013.

[2] Fielding R T. Architectural styles and the design of network-based software architectures[D]. Irvine: University of California, Irvine, 2000.

[3] Jones M, Bradley J, Sakimura N. JSON Web Token (JWT)[S/OL]. RFC 7519, IETF, 2015\[2025-01-31\]. https://www.rfc-editor.org/info/rfc7519.

[4] Donovan A A A, Kernighan B W. Go 程序设计语言[M]. 李道兵, 高宇馨, 译. 北京: 机械工业出版社, 2017.

[5] Gin Web Framework. Gin 官方文档[EB/OL]. \[2025-01-31\]. https://gin-gonic.com/zh-cn/docs/.

[6] Evan You. Vue.js 官方文档[EB/OL]. \[2025-01-31\]. https://cn.vuejs.org/.

[7] Nuxt. Nuxt 文档[EB/OL]. \[2025-01-31\]. https://nuxt.com/docs.

[8] 黄健宏. Redis 设计与实现[M]. 北京: 机械工业出版社, 2014.

[9] 董崇杰. MySQL 数据库技术[M]. 北京: 清华大学出版社, 2022.

[10] Jinzhu. GORM 官方文档[EB/OL]. \[2025-01-31\]. https://gorm.io/zh_CN/docs/.

[11] Ant Design Vue. Ant Design Vue 组件库[EB/OL]. \[2025-01-31\]. https://antdv.com/components/overview-cn.

[12] PayPal. PayPal REST API 参考[EB/OL]. \[2025-01-31\]. https://developer.paypal.com/docs/api/overview/.

[13] 覃征. 电子商务概论[M]. 4 版. 北京: 高等教育出版社, 2015.

[14] 刘颖君. 跨境电子商务基础[M]. 北京: 电子工业出版社, 2020.